

ENGLISH FOR SOFTWARE ENGINEERS AND DEVELOPMENT TEAMS

Software Development English

Make the behavior clear. Make the limits explicit.

Phil Stilwell

© 2026 Phil Stilwell. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission from the author, except as permitted by law.



Eight lessons. Eleven extended conversations.

INSIDE THE BOOK

192 vocabulary entries / 128 reusable phrases
220 extended-dialogue turns / 11 transfer exchanges
Word banks and explained answer keys

Upper-intermediate to advanced (B2-C1). For software developers discussing interfaces, debugging, reviews, dependencies, tests, performance, distributed behavior, and releases.

2026 edition

Eight scenarios. Eight skills.

Eight engineering conversations follow work from an unclear interface contract to a release handoff. Practice requesting evidence, challenging assumptions, explaining tradeoffs, and distinguishing a tested result from an unverified expectation.

01 Clarifying interface contracts

4

Three nickname states, only two defined meanings | Distinguish omission, an empty string, and null while requesting an explicit interface decision before implementation.

02 Debugging from reproducible evidence

13

The empty date is a suspect, not yet the cause | Report a reproducible failure, expose a confounded comparison, and request a controlled test without overstating causation.

03 Giving actionable code-review feedback

22

A rename that changed the failure path | Anchor a review comment in a changed call location, its observable effect, and a specific correction and test.

04 Requesting dependency evidence

31

A direct upgrade with an indirect change | Request the resolved dependency version, affected call sites, and migration evidence before claiming upgrade compatibility.

05 Explaining what a test actually exercises

40

A green test that never sees rejection | Explain the limits of a success-only test double and distinguish a failure-path unit test from real-adapter integration coverage.

06 Comparing performance tradeoffs fairly

49

Fifteen milliseconds faster, forty megabytes over budget | Compare measured latency and memory under matched conditions while separating median improvement from budget compliance and production claims.

07 Discussing retries and shared state

58

The timeout did not answer the business question | Explain an uncertain request outcome and request a retry contract before risking a second reservation.

08 Handing over a release with explicit limits

67

R18 is deployed, but the release decision is not made | Hand over environment, flag state, migration limits, and decision ownership without conflating deployment with production readiness.

Communication field notes

3

Three more workplace conversations

76

Answers and explanations

85

Fast-access phrase pages

104

Vocabulary index

106

Sources and publication notes

114

Describe behavior before confidence.

A confident engineering statement names its evidence and its boundary. A test double is not the real dependency; a staging deployment is not a production release; a timeout does not establish that a request failed.

Separate contract from convention

An omitted field, an empty string, and null can mean different things. State what this interface actually promises rather than relying on how another system behaves. A proposed interpretation needs agreement before it becomes a contract.

"Omission leaves the nickname unchanged; the meaning of null is not specified."

Make disagreement actionable

Anchor review comments in a changed line, observable behavior, and a concrete request. Keep naming preferences separate from defects. A focused failure test can make the concern easier to discuss without turning the review into a judgment of its author.

"Moving this call outside the success branch also sends completion notices after failure."

Attach the measurement conditions

A median describes the middle of observed results, not the slowest users or production behavior. Compare implementations under the same stated conditions, and include resource budgets. A faster measured response does not settle an overall tradeoff.

"B lowers the median to twenty-five milliseconds but exceeds the memory budget."

Keep release stages distinct

Code can be deployed but disabled, tested in staging but not production, or approved for review but not release. Name the environment, flag state, unresolved compatibility check, and actual decision owner in the handoff.

"R18 is in staging with search disabled; Lee still owns the release decision."

PROFESSIONAL BOUNDARIES

All products, people, measurements, interfaces, and release processes are fictional. These scenarios teach professional English, not production operating instructions. Actual contracts, security requirements, engineering review, test environments, and change authorization take precedence. No exercise authorizes production changes or guarantees correctness, compatibility, reliability, or performance. Code-review and testing terminology is used without requiring a particular language or framework.

Three nickname states, only two defined meanings

A Read the situation

Distinguish omission, an empty string, and null while requesting an explicit interface decision before implementation.

A fictional profile interface accepts an optional nickname. Its current contract says an omitted nickname leaves the stored value unchanged, while an empty string clears it. A proposed client sends null. The meaning of null has not been specified. Client developer Maya and service developer Arun review the mismatch before either side implements a new interpretation. Optional describes whether the field may be absent; it does not settle what an explicit null means.

PEOPLE / Maya: Client developer | Arun: Service developer

ASK FOR THE MISSING RULE, NOT A GUESS

Developers may use the same familiar word for different behaviors. Repeat the exact input and resulting state before agreeing that an interface is clear. An explicit question about null avoids hiding a product decision inside an implementation detail.

Briefing checks

Circle one answer in each item. Use the briefing.

1. What does omitting nickname do under the current contract?

- | | |
|-------------------------------|--|
| A Rejects every update | B Leaves the stored value unchanged |
| C Clears it | D Writes the word null |

2. What does an empty string do?

- | | |
|----------------------------------|--------------------------------------|
| A Leaves it unchanged | B Sets an unspecified default |
| C Proves null is accepted | D Clears the nickname |

3. What remains undefined?

- | | |
|--|---------------------------------------|
| A The meaning of an explicit null | B Whether omission is possible |
| C Whether an empty string clears | D Whether nickname is a field |

[Answers and explanations: page 85.](#)

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

interface contract

Agreed rules for inputs, outputs, and observable behavior.

IN USE clarify the interface contract

application programming interface

Defined way software components communicate; abbreviated API.

IN USE document the API

request payload

Data carried by a request.

IN USE inspect the request payload

optional field

Field that may be absent under the contract.

IN USE omit an optional field

nullable field

Field whose allowed values include null under the contract.

IN USE declare a nullable field

omitted field

Field not included in the submitted data.

IN USE distinguish an omitted field

empty string

Text value containing zero characters.

IN USE send an empty string

null value

Explicit marker whose meaning depends on the contract.

IN USE define the null value

stored state

Value or condition currently retained by the system.

IN USE preserve stored state

partial update

Change affecting only specified parts of a record.

IN USE apply a partial update

default value

Value supplied when the governing rules call for one.

IN USE document the default value

validation rule

Condition an input must satisfy.

IN USE agree on a validation rule

Precision check

Optional and nullable answer different questions. Optional concerns presence; nullable concerns allowed values. The stated contract defines omission and an empty string, but it does not authorize treating null as either one.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

serialization

Conversion of data into a transportable representation.

IN USE inspect serialization behavior

deserialization

Conversion from transported data into usable values.

IN USE verify deserialization behavior

schema

Formal description of data structure and constraints.

IN USE revise the schema

field presence

Whether a particular field exists in the input.

IN USE check field presence

strong ETag

Representation validator compared strongly in an If-Match condition.

IN USE compare the strong ETag

backward compatibility

Ability to preserve supported behavior for earlier consumers.

IN USE assess backward compatibility

consumer

Component using another component's interface.

IN USE identify the consumer

producer

Component providing data or behavior through an interface.

IN USE align producer expectations

decimal arithmetic

Base-ten calculation with specified precision and rounding behavior.

IN USE use decimal arithmetic

contract test

Test checking agreed interactions across an interface boundary.

IN USE add a contract test

unspecified interface behavior

Input effect not defined by this interface contract.

IN USE resolve unspecified interface behavior

acceptance criterion

Specific condition used to judge whether work meets requirements.

IN USE state an acceptance criterion

Precision check

Use an input-to-effect sentence: If nickname is omitted, the stored value remains unchanged. Keep the proposed null behavior explicitly unresolved until the responsible parties agree, document it, and add the appropriate checks.

Phrases you can actually use

C Notice the language / 1 of 2

NAME THE BOUNDARY

"I want to confirm the profile interface contract."

DESCRIBE CLEARING

"An empty string clears the nickname."

EXPOSE THE GAP

"The contract does not specify what null means."

ASK FOR THE EFFECT

"What should the stored value be after that input?"

DESCRIBE ABSENCE

"An omitted nickname leaves the stored value unchanged."

IDENTIFY THE PROPOSAL

"The proposed client sends null."

SEPARATE TERMS

"Optional does not automatically mean nullable."

AVOID BORROWING A RULE

"I would not infer this from another API."

Does not automatically mean

Rejects an inference without rejecting the other person's whole proposal.

After that input

Connects a request value to observable stored state.

Would be

Marks a proposed consequence rather than an existing rule.

Phrases you can actually use

C Notice the language / 2 of 2

INSPECT THE ACTUAL DATA

"Can we check the request payload?"

KEEP A PROPOSAL PROVISIONAL

"Treating null as clear would be a new decision."

REQUEST DOCUMENTATION

"Please record that rule in the contract."

AVOID PREMATURE COMPLETION

"The null case is still unresolved."

STATE A CONDITION

"If the field is absent, the current value stays."

REQUEST AGREEMENT

"Can we agree on the intended behavior first?"

REQUEST A CHECK

"We need a test for each defined input state."

CLOSE PRECISELY

"We agree on omission and empty string, but not yet on null."

Not yet

Keeps an unresolved decision visible without implying permanent disagreement.

Each defined input state

Prevents a broad test claim based on only one example.

Can we agree

Invites a concrete decision about the missing behavior.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. Which comment identifies the real gap?

- A** Empty string and omission are identical
- B** Every API interprets null alike
- C** The null case needs an agreed meaning
- D** Optional proves null clears it

2. Which pair already has different defined effects in this contract?

- A** Null and a documented clearing command
- B** Omission and an empty string
- C** Two spellings of the same field name
- D** Two confirmed clearing commands

3. What would justify calling null a clearing command?

- A** The developer's preference alone
- B** The word optional alone
- C** An unrelated API example
- D** An explicit agreed contract change

4. Which test plan matches the discussion?

- A** Check each defined input state and resolve null before asserting its effect
- B** Test omission and assume all other inputs match
- C** Test only the client's display
- D** Treat null as proven compatible

Answers and explanations: [page 85](#).

Three nickname states, only two defined meanings

E Conversations / turns 1-10

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

WORD + PHRASE BANK

interface contract / validation rule / backward compatibility / contract test / acceptance criterion / empty string / request payload / field presence / omitted field / optional field

- 01 **Maya:** The form currently sends null when a user removes a nickname. Before I ship that, which input does this service actually define as clearing?
- 02 **Arun:** We should separate that from an ¹ _____. If nickname is absent from the request, the stored value stays unchanged. That part is already defined.
- 03 **Maya:** I treated optional as permission to send null. But a missing property and a present property with a null value aren't the same payload.
- 04 **Arun:** Exactly. An ² _____ may be absent. It does not tell us whether null is accepted or what effect null should have if it is accepted.
- 05 **Maya:** For the existing clearing action, then, I'd send a string with zero characters. Leaving nickname out would preserve the old value instead.
- 06 **Arun:** Yes, an ³ _____ clears the nickname. We should preserve that distinction in the examples so nobody silently changes the current behavior while updating the client.
- 07 **Maya:** I'll stop calling null an alternative clearing command. We haven't agreed on what it means here, even if another API uses it that way.
- 08 **Arun:** Correct. The ⁴ _____ needs a decision for that input. We could discuss a change, but discussion alone does not make the proposed meaning an existing guarantee.
- 09 **Maya:** The client object shows null, but I haven't captured what its serializer sends. Could that conversion drop the property before the request leaves?
- 10 **Arun:** Let us check the ⁵ _____ in a controlled example. We need to distinguish the client's internal value from the representation delivered to the service.

The conversation, continued

E Conversations / turns 11-20

Continue with the word bank on page 10.

- 11 **Maya:** I'll show the actual three wire examples separately: absent, empty string, and explicit null. Only the first two have defined outcomes so far.
- 12 **Arun:** That will make ⁶ _____ visible. For the first two, write the resulting stored value beside the input rather than using the vague label no nickname.
- 13 **Maya:** Would allowing null in the schema get ahead of the decision? I don't want that change to imply a stored-state rule nobody has approved.
- 14 **Arun:** Hold that change until the ⁷ _____ and intended effect are agreed. Allowing a value and defining its effect need to be consistent.
- 15 **Maya:** Once the rule is settled, I'll test the client-service interaction, not just whether the form looks empty on screen.
- 16 **Arun:** A ⁸ _____ can support that. Its expected result must come from the agreement, though; the test must not invent the missing null rule.
- 17 **Maya:** And I want the omission case protected. Solving the null question shouldn't silently turn an unchanged nickname into a cleared one for older clients.
- 18 **Arun:** That is a ⁹ _____ concern worth recording. We have not demonstrated compatibility merely by agreeing that the new client looks correct.
- 19 **Maya:** Then I'll report omission preserves, empty string clears, and null unresolved. My proposed null implementation stays pending until the interface decision.
- 20 **Arun:** Good. That gives us an explicit ¹⁰ _____ once the decision is made, instead of treating an ambiguous word as enough to approve the change.

ANSWERS / EXPLANATIONS: PAGE 86

1. omitted field | 2. optional field | 3. empty string | 4. interface contract | 5. request payload | 6. field presence | 7. validation rule | 8. contract test | 9. backward compatibility | 10. acceptance criterion

Three inputs, three separate statements

F Say it

Complete the interface summary using only the stated contract and the unresolved proposal.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

nullable / clears / unchanged / unspecified

Developer: "An omitted nickname leaves the value ¹ _____."

Reviewer: "An empty string ² _____ the nickname."

Developer: "The meaning of null remains ³ _____."

Reviewer: "Optional does not automatically mean ⁴ _____."

ANSWERS / EXPLANATIONS: PAGE 86

1. unchanged | 2. clears | 3. unspecified | 4. nullable

Rehearse with a fixed script

1. Read Maya's three input examples, then read Arun's matching contract statements. Keep omission, empty string, and null distinct.
2. Swap roles and repeat turns 9-16, stressing actual payload and agreed validation rule.
3. Compare your readback with turns 19-20: two defined outcomes, one unresolved input, and no premature implementation approval.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

The empty date is a suspect, not yet the cause

A Read the situation

Report a reproducible failure, expose a confounded comparison, and request a controlled test without overstating causation.

Developer Nia has a three-row import file containing one empty date. It fails, and the stack trace points to date parsing. A two-row file without that row succeeds. Colleague Ellis reviews the evidence. No controlled test has isolated the empty date as the cause: the successful file differs in both row count and content. They need a test in a suitable nonproduction environment that changes one relevant condition while preserving the others.

PEOPLE / Nia: Developer investigating the import | Ellis: Developer reviewing the evidence

TREAT A HYPOTHESIS AS USEFUL, NOT EMBARRASSING

Naming a likely cause can help investigation, provided its status remains clear. Distinguish I suspect from I have isolated. A colleague can challenge a comparison without dismissing the observation or the person who found it.

Briefing checks

Circle one answer in each item. Use the briefing.

1. Which input fails?

- A** Every file with any date
- B** A three-row file containing an empty date
- C** Every three-row file
- D** A two-row file with all dates empty

2. What does the successful comparison change?

- A** Nothing relevant
- B** Both row count and content
- C** Only the date format
- D** Only the environment

3. What does the stack trace establish?

- A** The empty field is conclusively the root cause
- B** The database is corrupt
- C** Production is safe to test
- D** The observed failure path reaches date parsing

Answers and explanations: page 87.

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

reproduction

Repeatable demonstration of an observed failure.

IN USE obtain a reproduction

reproducible example

Input and conditions that consistently show the behavior.

IN USE share a reproducible example

stack trace

Recorded chain of calls associated with an error.

IN USE inspect the stack trace

parser

Component interpreting structured input.

IN USE inspect the parser

date parsing

Conversion of date text into a usable representation.

IN USE investigate date parsing

input fixture

Prepared input used in a test.

IN USE preserve the input fixture

hypothesis

Proposed explanation requiring evidence.

IN USE test a hypothesis

root cause

Underlying cause established through investigation.

IN USE isolate the root cause

confounding variable

Additional changed factor obscuring a comparison.

IN USE remove a confounding variable

controlled comparison

Comparison holding relevant conditions fixed except the tested change.

IN USE run a controlled comparison

optimistic concurrency

Detecting intervening changes when a conditional update is attempted.

IN USE enforce optimistic concurrency

rounding mode

Rule for selecting a representable value when a result must be rounded.

IN USE specify the rounding mode

Precision check

The two-row success is useful evidence, but it changes two things at once. It does not distinguish an empty-date effect from other differences associated with the removed row or the changed row count.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

failure signature

Identifying pattern of an observed failure.

IN USE compare the failure signature

exception

Signal that interrupts normal execution under specified conditions.

IN USE capture the exception

batched lookup

Retrieving data for several keys in one grouped operation.

IN USE replace repeated calls with a batched lookup

N+1 query pattern

One initial query followed by one related query per loaded item.

IN USE identify an N+1 query pattern

empty field

Present field containing no supplied value.

IN USE isolate the empty field

malformed input

Data not conforming to the required structure or rules.

IN USE identify malformed input

deterministic behavior

Behavior producing the same result under the same relevant conditions.

IN USE check deterministic behavior

environment parity

Relevant similarity between execution environments.

IN USE establish environment parity

regression

Previously working behavior broken by a later change.

IN USE investigate a regression

instrumentation

Added observation mechanisms such as logs or measurements.

IN USE improve instrumentation

causal claim

Statement that one condition produces an effect.

IN USE qualify a causal claim

nonproduction environment

Environment not serving the live production workload.

IN USE use a nonproduction environment

Precision check

A stack trace helps locate where the failure surfaced. It is not automatically a root-cause analysis. Propose a controlled comparison with the same row count and other content while varying the suspect date condition.

Phrases you can actually use

C Notice the language / 1 of 2

REPORT THE FAILING INPUT

"The three-row file fails during import."

NAME THE OBSERVATION

"The stack trace reaches date parsing."

QUALIFY THE CONCLUSION

"I suspect the empty date, but have not isolated it."

ASK FOR CONTROL

"Can we hold the other fields constant?"

REPORT THE DISTINGUISHING DETAIL

"One row contains an empty date."

REPORT THE COMPARISON

"The two-row file succeeds."

EXPOSE THE CONFOUND

"We changed both row count and content."

PROPOSE A NARROW CHANGE

"Change only the suspect date condition."

I suspect

Gives a working explanation an explicitly provisional status.

Have not isolated

Identifies the missing causal evidence.

Both ... and

Makes the two changing factors visible.

Phrases you can actually use

C Notice the language / 2 of 2

PRESERVE EVIDENCE

"Keep the original failing fixture."

ASK FOR REPEATABILITY

"Does the same fixture fail consistently?"

NAME THE SETTING

"Run the comparison in a suitable nonproduction environment."

REPORT A LIMIT

"We do not have the controlled result yet."

AVOID A UNIVERSAL CLAIM

"We have not shown that every empty date fails."

COMPARE THE OUTCOME

"Check whether the failure signature stays the same."

SEPARATE LOCATION AND CAUSE

"The trace points to parsing, not yet to a proven cause."

CLOSE THE HANDOFF

"Share the fixture, conditions, trace, and next comparison."

Hold constant

Requests an unchanged comparison condition.

Not yet

Limits the conclusion without discarding the evidence.

Whether

Introduces a check whose result is genuinely unresolved.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. Which conclusion is supported?

- A** The root cause is proven
- B** The observed failing path reaches date parsing
- C** All empty dates always fail
- D** Row count has no effect

2. Which proposed comparison is strongest?

- A** Change date, encoding, and parser version together
- B** Keep three rows and other content fixed while varying the suspect date
- C** Delete two more rows and change the environment
- D** Compare unrelated files

3. Which phrase avoids overstating evidence?

- A** The empty date is conclusively responsible
- B** The trace proves every input rule
- C** The successful file eliminates all other causes
- D** The empty date is a working hypothesis

4. What should the handoff preserve?

- A** The failing fixture, conditions, and trace
- B** Only the developer's confidence
- C** A rewritten file without its original data
- D** A production change without authorization

Answers and explanations: [page 87](#).

The empty date is a suspect, not yet the cause

E Conversations / turns 1-10

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

WORD + PHRASE BANK

deterministic behavior / reproducible example / hypothesis / confounding variable /
nonproduction environment / stack trace / failure signature / input fixture / causal claim /
controlled comparison

- 01 **Nia:** The three-row import failed, and the trace reaches date parsing. I've attached the input, including the empty date in row two.
- 02 **Ellis:** Let's make that a ¹ _____ before calling the cause settled. Can another run with the same input and conditions produce the same failure?
- 03 **Nia:** I haven't established that yet. I removed row two and the import passed, which made me suspect the empty date.
- 04 **Ellis:** That's a useful ² _____, not a conclusion. Removing the row changed its contents and the number of rows together.
- 05 **Nia:** I wrote that the empty date causes the error. You're asking me to narrow that to what the comparison actually shows.
- 06 **Ellis:** Yes. Row count is a ³ _____ in this comparison. We don't know whether it matters, but we haven't held it fixed.
- 07 **Nia:** Then I'll keep three rows and change only that date to a valid value, with the other data and settings unchanged.
- 08 **Ellis:** That gives us a ⁴ _____. Keep both files so we can inspect exactly what changed rather than rely on a description.
- 09 **Nia:** Should I reduce the file further first? Two rows would be easier to send around, but that version currently passes.
- 10 **Ellis:** Keep the failing ⁵ _____ intact. A smaller passing file isn't a smaller reproduction of the failure we're investigating.

The conversation, continued

E Conversations / turns 11-20

Continue with the word bank on page 19.

- 11 **Nia:** The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?
- 12 **Ellis:** It could. The ⁶ _____ records the observed call path; it doesn't establish which input condition first caused the problem.
- 13 **Nia:** I'll preserve the error type and location with the run details. If a later run fails differently, we shouldn't count it as the same result.
- 14 **Ellis:** Exactly. Compare the ⁷ _____ as well as pass or fail. Otherwise a changed exception could hide a different defect.
- 15 **Nia:** And I'll rerun each controlled input. A single failure followed by a single success doesn't establish repeatability.
- 16 **Ellis:** Right. Check ⁸ _____ rather than assuming it. Record variations if the same input doesn't reliably produce the same result.
- 17 **Nia:** I'll use the approved test environment, not try these files against a live customer import to get a quick answer.
- 18 **Ellis:** Use the suitable ⁹ _____ and keep its configuration in the record. We need evidence without changing production data.
- 19 **Nia:** My update will say the empty date is suspected, the successful comparison changed two factors, and a controlled repeat test is pending.
- 20 **Ellis:** That keeps the ¹⁰ _____ at the right strength. Bring back the actual results; don't mark the hypothesis confirmed before the comparison is run.

ANSWERS / EXPLANATIONS: PAGE 88

1. reproducible example | 2. hypothesis | 3. confounding variable | 4. controlled comparison | 5. input fixture | 6. stack trace | 7. failure signature | 8. deterministic behavior | 9. nonproduction environment | 10. causal claim

Report the evidence without upgrading the conclusion

F Say it

Complete the debugging handoff. The controlled test has not yet produced a result.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

two / hypothesis / three / constant

Developer: "The failing file contains ¹ _____ rows."

Reviewer: "The successful file contains ² _____ rows."

Developer: "The empty date remains a ³ _____."

Reviewer: "Keep the other conditions ⁴ _____ in the next comparison."

ANSWERS / EXPLANATIONS: PAGE 88

1. three | 2. two | 3. hypothesis | 4. constant

Rehearse with a fixed script

1. Read the failed-file report and the two-row comparison. Stress which facts were observed and which cause was only suspected.
2. Swap roles and repeat turns 5-10, keeping row count constant in the proposed test.
3. Check your closing report against turns 17-20: nonproduction setting, pending comparison, and no confirmed causal claim.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

A rename that changed the failure path

03

A Read the situation

Anchor a review comment in a changed call location, its observable effect, and a specific correction and test.

A pull request is titled Rename export helper. The diff also moves the completion-notification call outside the success branch. Failed exports therefore notify users as though export completed. Reviewer Priya accepts the naming change but raises the behavior change with author Tomas. The review should request restoration of success-only notification and a failure-path test. The acceptable rename does not justify approving the unintended notification behavior.

PEOPLE / Priya: Code reviewer | Tomas: Pull-request author

CRITIQUE THE CHANGE, NOT THE AUTHOR

A useful review points to behavior and asks for a verifiable change. Acknowledge acceptable parts without letting praise obscure a defect. Separating naming from functionality lets the author respond to a concrete concern instead of defending personal competence.

Briefing checks

Circle one answer in each item. Use the briefing.

1. What is the pull request titled?

- | | |
|--------------------------------------|----------------------------------|
| A Release the importer | B Rename export helper |
| C Rewrite notification policy | D Remove all export tests |

2. What additional change appears in the diff?

- | | |
|---|--|
| A Notifications are removed entirely | B Notification moves outside the success branch |
| C The export always succeeds | D The failure branch is proven unreachable |

3. What part does Priya accept?

- | | |
|---------------------------------------|-----------------------------------|
| A The false success message | B Missing failure coverage |
| C Automatic production release | D The naming change |

Answers and explanations: [page 89](#).

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

pull request

Proposed code change submitted for review; abbreviated PR.

IN USE review the pull request

diff

Display of changes between versions.

IN USE inspect the diff

review comment

Feedback attached to a proposed change.

IN USE write a review comment

success branch

Execution path entered when the relevant operation succeeds.

IN USE keep the call in the success branch

failure path

Execution path taken when an operation fails.

IN USE cover the failure path

call site

Location where a function or operation is invoked.

IN USE identify the call site

control flow

Order and conditions under which code executes.

IN USE trace the control flow

side effect

Observable effect beyond a function's returned value.

IN USE examine the side effect

completion notification

Message indicating an operation has finished successfully here.

IN USE gate the completion notification

behavior change

Alteration in what the system observably does.

IN USE flag the behavior change

refactor

Internal restructuring intended to preserve observable behavior.

IN USE separate a refactor from a behavior change

rename

Change to an identifier's name.

IN USE approve the rename

Precision check

A rename can be acceptable while the same diff contains a blocking defect. Moving the notification outside the success branch changes observable behavior, so it is not merely a naming preference or a behavior-preserving refactor.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

regression test

Test guarding against recurrence of a known defect.

IN USE add a regression test

assertion

Test statement checking an expected condition.

IN USE strengthen the assertion

blocking concern

Issue that must be resolved before approval under the review process.

IN USE explain the blocking concern

nonblocking suggestion

Improvement that does not prevent approval under the process.

IN USE label a nonblocking suggestion

review scope

Set of changes and concerns covered by a review.

IN USE clarify review scope

intent

Purpose the author means a change to achieve.

IN USE confirm the intent

observable behavior

Effect visible to a user or another component.

IN USE preserve observable behavior

false success

Indication of success when the operation did not succeed.

IN USE prevent false success

guard condition

Condition controlling whether an action may occur.

IN USE restore the guard condition

patch revision

Updated version of a proposed change.

IN USE request a patch revision

approval

Explicit acceptance at the relevant review stage.

IN USE withhold approval pending correction

verification evidence

Results supporting a claim that a change behaves as intended.

IN USE provide verification evidence

Precision check

Make the request testable: restore success-only notification, then add a check that a failed export does not produce a completion notice. Do not substitute a general request to improve code quality for the specific observed issue.

Phrases you can actually use

C Notice the language / 1 of 2

03

ACKNOWLEDGE THE ACCEPTABLE PART

"The helper name is clear."

DESCRIBE THE CONSEQUENCE

"A failed export can now send a completion notice."

ASK ABOUT INTENT

"Was this control-flow change intentional?"

MAKE A CONCRETE REQUEST

"Please restore the success-only condition."

LOCATE THE CONCERN

"This notification call has moved outside the success branch."

SEPARATE PREFERENCE AND DEFECT

"My concern is behavior, not the naming choice."

STATE THE EXPECTED BOUNDARY

"The completion notice belongs on the success path."

REQUEST EVIDENCE

"Add a test for a failed export."

This call

Anchors the concern to a particular location in the change.

Can now

Identifies a newly possible behavior rather than a style preference.

Was ... intentional

Asks about purpose without assuming carelessness.

Phrases you can actually use

C Notice the language / 2 of 2

SPECIFY THE ASSERTION

"The failure case should not send a completion notification."

CLARIFY REVIEW STATUS

"I cannot approve this behavior yet."

DISTINGUISH SCOPE

"The diff does more than the title suggests."

KEEP VERIFICATION SEPARATE

"I will review the revised behavior and test."

AVOID PERSONAL CRITICISM

"This change introduces a false-success path."

LIMIT THE OBJECTION

"I am not asking you to undo the rename."

REQUEST A REVISION

"Please update the patch and its description."

CLOSE CONSTRUCTIVELY

"The rename can stay once the notification issue is resolved."

Not ... but

Separates the accepted naming issue from the behavioral objection.

Should not

States the negative assertion needed in the failure test.

Once

Makes approval conditional on a specific unresolved correction.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. Which comment is actionable?

- A** Restore the success-only notification and test that failure sends no completion notice
- B** Your code is careless
- C** Make this nicer
- D** The name is fine, so everything is approved

2. Why is this not only a refactor?

- A** Observable notification behavior changes
- B** The helper has a new name
- C** Review comments are present
- D** The title is short

3. What should the failure test assert?

- A** Production release is approved
- B** No completion notice is sent
- C** A completion notice always appears
- D** Only the helper spelling is correct

4. Which statement preserves Priya's position?

- A** The rename is acceptable; the notification behavior needs correction
- B** Every line must be reverted
- C** The author cannot write software
- D** The test can be omitted because the rename is small

Answers and explanations: [page 89](#).

A rename that changed the failure path

E Conversations / turns 1-10

03

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

WORD + PHRASE BANK

verification evidence / side effect / patch revision / control flow / rename / false success / failure path / review comment / regression test / review scope

- 01 **Priya:** The helper name is clearer. Before I approve, though, can we look at the notification call you moved below the success branch?
- 02 **Tomas:** I titled this as a ¹ _____. Are you concerned about the identifier or something the new position changes?
- 03 **Priya:** The position. A failed export can now reach that call, so the user could receive a completion notice for an unsuccessful export.
- 04 **Tomas:** I see the ² _____ change. I pulled the call out while cleaning up the method, but that removes the success condition.
- 05 **Priya:** Keep the clearer name. Restore the guard around the notification; that's the behavior I'm blocking on, not my naming preference.
- 06 **Tomas:** Understood. The notification is a ³ _____ we need to preserve correctly. I'll keep it dependent on a successful export.
- 07 **Priya:** Please add a failing-export case too. It should verify that no completion notice is sent, not merely that export was attempted.
- 08 **Tomas:** So the ⁴ _____ needs a no-notification expectation. A test that only checks the helper call could pass with this defect still present.
- 09 **Priya:** Exactly. I'll point to the moved call and explain that consequence in the review so you don't have to infer the requested change.
- 10 **Tomas:** That makes the ⁵ _____ actionable. I can address the changed behavior without reopening the whole naming discussion.

The conversation, continued

E Conversations / turns 11-20

Continue with the word bank on page 28.

- 11 **Priya:** Also check the description against the final patch. Right now the title makes this sound like a purely mechanical change.
- 12 **Tomas:** I'll clarify the ⁶ _____. A small diff and a rename title don't establish that the actual behavior stayed the same.
- 13 **Priya:** Would you agree the current version isn't behavior-preserving? The failed export and the completion notice contradict each other.
- 14 **Tomas:** Yes. That ⁷ _____ is a defect, not a style disagreement. I won't describe the current patch as an approved refactor.
- 15 **Priya:** Once the guard and failure assertion are in place, send the updated change back with the relevant test result.
- 16 **Tomas:** I'll submit a ⁸ _____ for another review. Your acceptance of the name isn't approval of the whole change.
- 17 **Priya:** Make sure the new check would fail if that notification moved outside the success condition again. Otherwise it won't protect this behavior.
- 18 **Tomas:** I'll verify the ⁹ _____ catches that specific recurrence. Simply executing the failure branch isn't enough if the assertion misses the notice.
- 19 **Priya:** Good. My review remains blocked on corrected behavior and its check. I'm happy with the name; those are separate decisions.
- 20 **Tomas:** I'll return with ¹⁰ _____ for the correction. We can resolve the blocking comment after you've reviewed that, not from this conversation alone.

ANSWERS / EXPLANATIONS: PAGE 90

1. rename | 2. control flow | 3. side effect | 4. failure path | 5. review comment | 6. review scope | 7. false success | 8. patch revision | 9. regression test | 10. verification evidence

Turn a vague objection into a review request

F Say it

Complete the review summary while preserving the accepted rename and unresolved behavior issue.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

failure / approval / success / name

Reviewer: "The helper ¹ _____ is acceptable."

Author: "The notification must remain on the ² _____ path."

Reviewer: "Add a test for the ³ _____ path."

Author: "The current feedback is not yet ⁴ _____."

ANSWERS / EXPLANATIONS: PAGE 90

1. name | 2. success | 3. failure | 4. approval

Rehearse with a fixed script

1. Read the review once, keeping acceptance of the name separate from the blocking behavior concern.
2. Swap roles for turns 5-10. Stress success-only notification and no notification after failure.
3. Repeat the final four turns. Include the regression check and pending review rather than announcing approval.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

A direct upgrade with an indirect change

04

A Read the situation

Request the resolved dependency version, affected call sites, and migration evidence before claiming upgrade compatibility.

A fictional application uses ParcelKit directly. ParcelKit in turn uses TextCore. The proposed ParcelKit upgrade changes the resolved TextCore version. A change note marks a parsing entry point as deprecated. Developers Leila and Ben have not established whether the application or its dependencies use that entry point, whether it has been removed, or whether the replacement is compatible. The packages are not assumed to follow any particular versioning convention.

PEOPLE / Leila: Developer proposing the upgrade | Ben: Developer checking dependency impact

ASK WHAT CHANGED BELOW THE HEADLINE

An upgrade can affect more than the package named in the request. Asking for the resolved dependency tree and affected calls is a practical evidence request, not resistance to maintenance. Avoid translating deprecated into removed or compatible without checking the actual contract.

Briefing checks

Circle one answer in each item. Use the briefing.

1. Which dependency does the application use directly?

- | | |
|------------------------|---------------------------------|
| A Neither | B ParcelKit |
| C TextCore only | D Both are proven direct |

2. What changes indirectly?

- | | |
|--|---|
| A Every public interface | B A confirmed production failure |
| C The resolved TextCore version | D The application name |

3. What does the note establish?

- | | |
|--|--|
| A A parsing entry point is deprecated | B The replacement is fully compatible |
| C The application definitely uses it | D The entry point has definitely been removed |

Answers and explanations: [page 91](#).

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

direct dependency

Package explicitly used or declared by the application here.

IN USE upgrade a direct dependency

transitive dependency

Package brought in through another dependency.

IN USE inspect a transitive dependency

dependency tree

Structure showing how packages depend on one another.

IN USE inspect the dependency tree

resolved version

Specific version selected after dependency resolution.

IN USE record the resolved version

lockfile

File recording resolved dependency selections for reproducibility.

IN USE inspect the lockfile

version constraint

Rule limiting which dependency versions may be selected.

IN USE check a version constraint

release note

Published description of changes in a release.

IN USE read the release note

deprecation

Notice that an interface is discouraged or scheduled for replacement.

IN USE investigate a deprecation

removal

Elimination of a previously available interface or behavior.

IN USE verify removal separately

migration guide

Instructions describing transition to a changed interface.

IN USE consult the migration guide

entry point

Named interface through which functionality is invoked.

IN USE locate the parsing entry point

call site

Location that invokes a particular interface.

IN USE search for affected call sites

Precision check

Direct describes the application-to-ParcelKit relationship. Transitive describes the application-to-TextCore relationship through ParcelKit. The dependency actually selected matters, not just the version written in the top-level upgrade request.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

public interface

Supported surface exposed for other components to use.

IN USE compare the public interface

breaking change

Change that can invalidate previously supported use.

IN USE identify a breaking change

compatibility

Ability to work with the relevant existing consumers and behavior.

IN USE verify compatibility

semantic versioning

Version convention relating version numbers to public-interface changes.

IN USE confirm semantic versioning policy

major version

Leading version component under a stated convention.

IN USE interpret the major version

minor version

Version component whose meaning depends on the adopted convention.

IN USE check minor-version guarantees

patch version

Version component commonly used for fixes under a stated convention.

IN USE verify patch-version policy

pin

Fixed dependency selection restricting version movement.

IN USE review the version pin

changelog

Maintained record of notable changes between versions.

IN USE compare the changelog

replacement interface

Interface intended to supersede another.

IN USE assess the replacement interface

resolution

Process selecting concrete dependency versions.

IN USE reproduce dependency resolution

upgrade evidence

Records and checks supporting an upgrade assessment.

IN USE assemble upgrade evidence

Precision check

Deprecated does not itself mean removed, unused, or compatible with a replacement. Version numbers provide guarantees only under the package policy that actually applies. Request usage and migration evidence before declaring the upgrade safe.

Phrases you can actually use

C Notice the language / 1 of 2

IDENTIFY THE DIRECT DEPENDENCY

"The application uses ParcelKit directly."

NAME THE ADDITIONAL CHANGE

"The resolved TextCore version also changes."

ASK FOR THE RELATIONSHIP

"Can you show the dependency tree?"

AVOID AN UNSUPPORTED UPGRADE

"Deprecated does not by itself mean removed."

IDENTIFY THE INDIRECT DEPENDENCY

"TextCore comes through ParcelKit."

REQUEST THE ACTUAL SELECTION

"Which version does the lockfile record?"

QUOTE THE NARROW FINDING

"The note marks the parsing entry point as deprecated."

ASK ABOUT USAGE

"Do we have any affected call sites?"

Comes through

Expresses an indirect dependency relationship in ordinary speech.

Also changes

Exposes a second effect hidden by the top-level upgrade description.

By itself

Limits what the deprecation label establishes.

Phrases you can actually use

C Notice the language / 2 of 2

INCLUDE INDIRECT CALLERS

"Check whether ParcelKit invokes it as well."

SEPARATE AVAILABILITY AND BEHAVIOR

"An available replacement is not automatically compatible."

REQUEST VERIFICATION

"Which checks exercise the affected parsing behavior?"

STATE THE PENDING WORK

"Usage and replacement behavior still need review."

REQUEST MIGRATION DETAIL

"What does the migration guide say about the replacement?"

QUALIFY THE VERSION INFERENCE

"We need the package's actual versioning policy."

KEEP THE DECISION OPEN

"We do not have enough evidence to claim compatibility."

CLOSE THE REQUEST

"Please attach the resolved versions and the relevant findings."

Whether

Leaves actual usage unresolved.

Not automatically

Challenges a shortcut inference without claiming the opposite.

Actual

Directs attention to the selected versions and applicable policy.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. Which statement is accurate?

- A TextCore is transitive through ParcelKit
- B TextCore cannot affect the application
- C ParcelKit is only transitive
- D All transitive dependencies are unused

2. What should be checked first about the selected version?

- A The latest version mentioned in a blog
- B The resolved dependency record
- C Only the pull-request title
- D An unrelated package's policy

3. What does deprecated not establish?

- A That the interface deserves attention
- B That the entry point has been removed
- C That a transition may need review
- D That a note exists

4. Which compatibility claim is justified now?

- A The replacement is guaranteed identical
- B Every minor upgrade is safe
- C No call-site review is needed
- D Compatibility remains unverified

Answers and explanations: [page 91](#).

A direct upgrade with an indirect change

E Conversations / turns 1-10

04

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

WORD + PHRASE BANK

call site / deprecation / semantic versioning / compatibility / migration guide / upgrade evidence / dependency tree / transitive dependency / public interface / lockfile

- 01 **Leila:** The ParcelKit upgrade also selects a different TextCore version. The request title doesn't show that indirect change, but the resolved list does.
- 02 **Ben:** Then we need to review the ¹ _____ change as well. The application uses ParcelKit directly, but that does not make TextCore irrelevant to its behavior.
- 03 **Leila:** I can attach the package constraints and actual selected versions. Which record do you need to see what this build will really use?
- 04 **Ben:** Please include the ² _____ and the relevant dependency relationship. A constraint can allow several versions, while the resolved record tells us which one this change actually selects.
- 05 **Leila:** The release notes mark a parsing entry point deprecated. I don't yet know whether our code or ParcelKit's internals call it.
- 06 **Ben:** Keep that as a ³ _____ finding, not a removal finding. The note deserves investigation, but its label alone does not tell us whether this application is affected.
- 07 **Leila:** I'll remove my claim that the upgrade deletes a parser we use. Deprecation, removal, and our usage are three different questions.
- 08 **Ben:** Correct. Start by locating any relevant ⁴ _____, including indirect calls through ParcelKit. An absence of direct application calls would not automatically rule out an indirect dependency on that behavior.
- 09 **Leila:** The replacement has a similar name. Before I switch anything, where should we check whether its inputs and error behavior actually match?
- 10 **Ben:** Consult the ⁵ _____ first. Similar names do not establish identical inputs, outputs, error behavior, or handling of edge cases. We need the relevant documented differences.

The conversation, continued

E Conversations / turns 11-20

04

Continue with the word bank on page 37.

- 11 **Leila:** The version looks familiar, but we haven't checked either package's versioning policy. I shouldn't infer its compatibility promise from the number alone.
- 12 **Ben:** Then do not assume ⁶ _____ guarantees. We can check the actual policy, but a familiar number pattern is not evidence that its promised compatibility rules apply.
- 13 **Leila:** I'll trace ParcelKit to TextCore in the review notes. That should explain why a package we don't call directly still matters to this upgrade.
- 14 **Ben:** Good. The ⁷ _____ will help explain why TextCore belongs in the discussion even though the request is named after ParcelKit.
- 15 **Leila:** The existing suite passes, but I haven't shown that it exercises the affected parsing entry point. What evidence should the review request?
- 16 **Ben:** Ask which checks exercise the relevant ⁸ _____. A passing suite cannot establish coverage of a path it never executes.
- 17 **Leila:** I'll name that coverage gap instead of saying all tests passed therefore compatible. A green suite could simply miss the changed path.
- 18 **Ben:** Exactly. We need ⁹ _____ tied to this change. It should show what was checked and what remains unknown, not just that an upgrade command succeeded.
- 19 **Leila:** My revised summary will separate resolved versions, deprecation status, usage, and replacement behavior. The last two still need investigation.
- 20 **Ben:** That is accurate. Keep ¹⁰ _____ unverified for now. Once those findings are available, we can discuss the upgrade on evidence rather than infer its impact from the headline.

ANSWERS / EXPLANATIONS: PAGE 92

1. transitive dependency | 2. lockfile | 3. deprecation | 4. call site | 5. migration guide | 6. semantic versioning | 7. dependency tree | 8. public interface | 9. upgrade evidence | 10. compatibility

Keep the dependency chain and uncertainty intact

F Say it

Complete the upgrade handoff without turning deprecation into confirmed removal or compatibility.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

deprecated / unverified / transitive / direct

Developer: "ParcelKit is the ¹ _____ dependency."

Reviewer: "TextCore is the ² _____ dependency."

Developer: "The parsing entry point is marked ³ _____."

Reviewer: "Replacement compatibility remains ⁴ _____."

ANSWERS / EXPLANATIONS: PAGE 92

1. direct | 2. transitive | 3. deprecated | 4. unverified

Rehearse with a fixed script

1. Read the dependency discussion and distinguish the requested ParcelKit change from the selected TextCore version.
2. Swap roles and repeat turns 5-12. Keep deprecated, removed, used, and compatible separate.
3. Check the final readback against turns 17-20. Usage and compatibility must remain unresolved until evidence is supplied.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

A green test that never sees rejection

A Read the situation

Explain the limits of a success-only test double and distinguish a failure-path unit test from real-adapter integration coverage.

05

A save-operation unit test uses a storage double that always succeeds and checks the success message. A previous bug showed success after storage rejected the save. The current test never produces rejection, and the real storage adapter is untested. Developers Owen and Farah need to configure a rejection case, assert that false success does not appear, and keep real-adapter integration coverage as a separate unresolved concern.

PEOPLE / Owen: Developer maintaining the test | Farah: Developer reviewing coverage

A PASSING RESULT NEEDS A SCOPE STATEMENT

Green is a useful status, but it is not a complete explanation. Name the behavior the test actually exercises and the dependencies it replaces. This allows a colleague to request missing coverage without denying the value of existing tests.

Briefing checks

Circle one answer in each item. Use the briefing.

1. What does the storage double currently do?

- A Always succeeds
- C Uses the real adapter

- B Rejects every request
- D Alternates unpredictably

2. What did the previous bug show?

- A A missing helper name
- C Success after storage rejected the save

- B A proven adapter timeout
- D Failure after a confirmed save

3. What remains separately untested?

- A The existence of the double
- C The real storage adapter

- B Every application feature
- D Whether the test shows a success message

Answers and explanations: [page 93](#).

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

unit test

Focused test of a small behavior or component.

IN USE extend the unit test

test double

Substitute for a real dependency during testing.

IN USE configure the test double

stub

Test substitute supplying predetermined responses.

IN USE use a success stub

mock

Test substitute that can record or check interactions.

IN USE inspect mock interactions

fixture

Prepared state or data used by a test.

IN USE arrange the fixture

success response

Result indicating an operation succeeded.

IN USE return a success response

rejection

Failed outcome reported by the storage operation here.

IN USE simulate a rejection

exception

Error signal raised during execution under the relevant model.

IN USE configure an exception

assertion

Explicit check of an expected test condition.

IN USE add a negative assertion

negative assertion

Check that an unwanted event or state does not occur.

IN USE verify a negative assertion

test isolation

Separation of a tested unit from unrelated dependencies.

IN USE preserve test isolation

integration test

Test of interaction between connected components.

IN USE add an integration test

Precision check

A double that always succeeds can support a success-path assertion, but it cannot exercise rejection handling. Configure a failing response and check the user-visible consequence relevant to the known bug.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

adapter

Component translating between an application and an external interface.

IN USE test the storage adapter

branch coverage

Measure of which decision outcomes a test set executes.

IN USE examine branch coverage

happy path

Expected successful execution path.

IN USE cover the happy path

error path

Execution path handling an unsuccessful operation.

IN USE exercise the error path

false negative

Test result that misses a defect which is present.

IN USE avoid a false negative

test oracle

Rule or reference used to judge the expected result.

IN USE define the test oracle

arrange

Prepare the conditions for a test.

IN USE arrange a rejection case

act

Execute the behavior being tested.

IN USE act on the save request

assert

Check the observed result against the expectation.

IN USE assert no false success

interaction verification

Check of how a component called its dependencies.

IN USE separate interaction verification

behavioral verification

Check of the externally observable result.

IN USE perform behavioral verification

coverage gap

Relevant behavior not exercised by the current tests.

IN USE report the coverage gap

Precision check

The rejection unit test can verify application behavior against a simulated storage outcome. It does not establish that the real adapter produces that outcome correctly or that the actual integration behaves the same way.

Phrases you can actually use

C Notice the language / 1 of 2

DESCRIBE THE CURRENT SETUP

"The storage double always succeeds."

RECALL THE DEFECT

"The bug showed success after a rejected save."

REQUEST A CONFIGURED OUTCOME

"Make the double reject in a separate case."

PRESERVE EXISTING COVERAGE

"Keep the successful-save case too."

NAME THE TESTED PATH

"This test exercises the happy path."

IDENTIFY THE GAP

"The current test never triggers rejection."

SPECIFY THE CHECK

"Assert that no success message appears after rejection."

SEPARATE CALL AND RESULT

"A storage call alone does not prove the right user message."

Always

Exposes a fixture that cannot produce the missing failure condition.

After rejection

Connects the assertion to the relevant event sequence.

Separate case

Preserves success coverage while adding failure coverage.

Phrases you can actually use

C Notice the language / 2 of 2

NAME THE SUBSTITUTE

"This is a double, not the real adapter."

REQUEST ANOTHER TEST LEVEL

"The real adapter needs separate integration coverage."

STATE THE OPEN QUESTION

"We have not verified the real adapter."

REQUEST EVIDENCE

"Please include the configured outcome and assertion."

LIMIT THE CONCLUSION

"The result applies to the simulated rejection case."

AVOID A BLANKET CLAIM

"Green tests do not establish all storage behavior."

ASK ABOUT EXPECTATIONS

"What observable result should this case check?"

CLOSE ACCURATELY

"The new unit case can close one gap, not every storage risk."

Alone

Limits what a dependency-call assertion can demonstrate.

Simulated

Marks the boundary between a double and real integration.

One gap

Avoids overstating the scope of the proposed improvement.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. Why does the current test miss the prior bug?

- A** Every unit test uses production storage
- B** Success messages cannot be tested
- C** Its double never rejects
- D** Its name contains save

2. Which addition targets the defect?

- A** Check only that storage is called
- B** Remove all assertions
- C** Configure rejection and assert no success message
- D** Rename the success test only

3. What does the rejection unit test not establish?

- A** Correct behavior of the real storage adapter
- B** Behavior under the configured double response
- C** Whether the assertion passes in that case
- D** Whether the fixture rejects

4. Which report is precise?

- A** Rejection cannot be simulated
- B** Success-path coverage exists; rejection and real-adapter coverage need work
- C** All storage behavior is verified
- D** A green test proves no bugs remain

Answers and explanations: [page 93](#).

A green test that never sees rejection

E Conversations / turns 1-10

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

05

WORD + PHRASE BANK

Interaction verification / behavioral verification / arrange / integration test / happy path / success response / negative assertion / test double / unit test / coverage gap

- 01 **Owen:** The save test passes. I was about to mark the old false-success bug covered, but the storage substitute always returns success.
- 02 **Farah:** Then this checks the ¹ _____. Where does the test make storage reject the save, which is what exposed the old bug?
- 03 **Owen:** It doesn't. I have no rejection case in this fixture, so the green result can't tell us whether we still ignore rejection.
- 04 **Farah:** Configure the ² _____ for that outcome in a separate case. Keep the successful-save case; it still checks useful behavior.
- 05 **Owen:** I could assert that storage was called once. Would that distinguish the corrected application from the old one?
- 06 **Farah:** Not by itself. Add a ³ _____ that the success message is absent after rejection. Both versions might call storage once.
- 07 **Owen:** Right, the old version called it but treated the rejected save as completed. The call count wouldn't catch that.
- 08 **Farah:** Exactly. ⁴ _____ answers whether the dependency was called as expected; it doesn't replace checking what the user is told afterward.
- 09 **Owen:** This still crosses the storage interface. Could I label it an integration check once the failure case is in place?
- 10 **Farah:** Not as evidence for the real adapter. An ⁵ _____ of that interaction needs the relevant actual components; your adapter remains replaced.

The conversation, continued

E Conversations / turns 11-20

Continue with the word bank on page 46.

- 11 **Owen:** Then the description should say simulated storage rejection and application response. We haven't exercised the actual adapter at all.
- 12 **Farah:** Keep that ⁶ _____ explicit. Better simulated coverage doesn't silently close a different untested boundary.
- 13 **Owen:** I'll prepare the rejected result, execute the save, then inspect the displayed message. The fixture should make the unsuccessful outcome obvious.
- 14 **Farah:** Yes: ⁷ _____ the rejection, act on the save, and assert no success indication. Reviewers should see that setup without hunting through defaults.
- 15 **Owen:** I'll retain the original check separately. It confirms that the application shows the intended message when the substitute reports a successful save.
- 16 **Farah:** That's a bounded claim about the ⁸ _____ case. The misleading part was describing it as proof of the complete storage workflow.
- 17 **Owen:** After adding rejection, I'll report exactly which outcome was simulated and which visible behavior was checked, alongside the real-adapter gap.
- 18 **Farah:** That is useful ⁹ _____. Report the assertion result, not a blanket statement that storage is fully tested.
- 19 **Owen:** The patch is still pending review. I'll attach both test results when they're available, without inventing a successful run in the handoff.
- 20 **Farah:** Good. A targeted ¹⁰ _____ can catch this application defect while leaving the real integration question clearly open.

ANSWERS / EXPLANATIONS: PAGE 94

1. happy path | 2. test double | 3. negative assertion | 4. Interaction verification | 5. integration test | 6. coverage gap | 7. arrange | 8. success response | 9. behavioral verification | 10. unit test

Name the test boundary

F Say it

Complete the review note about the current test and the proposed rejection case.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

rejection / untested / success / succeeds

Developer: "The existing double always ¹ _____."

Reviewer: "The new case must trigger ² _____."

Developer: "The assertion should prevent a false ³ _____ message."

Reviewer: "The real adapter remains ⁴ _____."

ANSWERS / EXPLANATIONS: PAGE 94

1. succeeds | 2. rejection | 3. success | 4. untested

Rehearse with a fixed script

1. Read the save-test exchange, stressing always succeeds when describing the original double.
2. Swap roles and repeat turns 5-12. Distinguish a no-success-message assertion from checking a dependency call.
3. Use the final four turns to rehearse a bounded test report. Include the simulated rejection and remaining real-adapter gap.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

Fifteen milliseconds faster, forty megabytes over budget

A Read the situation

Compare measured latency and memory under matched conditions while separating median improvement from budget compliance and production claims.

06

In a controlled benchmark using the same input and machine, implementation A has median latency of 40 milliseconds and memory use of 80 megabytes. B has median latency of 25 milliseconds and memory use of 160 megabytes. The memory budget is 120 megabytes. Developers Sora and Malik discuss the tradeoff. Tail latency and production behavior have not been measured. B reduces the measured median by 15 milliseconds, or 37.5 percent relative to A, but exceeds the memory budget by 40 megabytes.

PEOPLE / Sora: Developer presenting the benchmark | Malik: Developer reviewing the tradeoff

DO NOT HIDE A CONSTRAINT INSIDE A HEADLINE

Faster is incomplete when a resource budget matters. State the metric, conditions, comparison baseline, and constraint together. A colleague can acknowledge a genuine latency improvement while rejecting the claim that it settles the overall implementation decision.

Briefing checks

Circle one answer in each item. Use the briefing.

1. Which conditions are matched?

A Tail-latency percentiles

B Input and machine

C Production traffic and every user

D All future workloads

2. How far is B over the memory budget?

A 80 megabytes

B It is under budget

C 40 megabytes

D 15 megabytes

3. What is unmeasured?

A The budget difference

B Tail latency and production behavior

C The stated median values

D The stated memory values

Answers and explanations: [page 95](#).

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

benchmark

Structured measurement under specified conditions.

IN USE run a benchmark

latency

Time taken for a request or operation.

IN USE measure latency

median

Middle ordered value, or mean of the two middle values for an even sample.

IN USE report median latency

tail latency

Slower-end behavior in a latency distribution.

IN USE measure tail latency

percentile

Value below which a stated percentage of observations falls.

IN USE report a latency percentile

p95

Ninety-fifth percentile of a measured distribution.

IN USE distinguish p95 from the median

throughput

Amount of work completed per unit of time.

IN USE measure throughput separately

memory footprint

Amount of memory used under the stated conditions.

IN USE compare the memory footprint

resource budget

Allowed limit for a specified resource.

IN USE respect the resource budget

headroom

Remaining capacity below a stated limit.

IN USE calculate memory headroom

overrun

Amount by which usage exceeds a limit.

IN USE quantify the budget overrun

baseline

Reference implementation or measurement for comparison.

IN USE use A as the baseline

06

Precision check

B reduces the median from 40 to 25 milliseconds: a 15-millisecond difference and a 37.5 percent reduction relative to A. It uses twice A's measured memory, but doubling memory is not the same as doubling speed.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

relative reduction

Decrease expressed as a fraction of the original value.

IN USE calculate the relative reduction

absolute difference

Direct numerical subtraction between comparable measurements.

IN USE report the absolute difference

tradeoff

Gain in one property accompanied by a cost in another.

IN USE explain the tradeoff

workload

Type and amount of work applied to a system.

IN USE describe the workload

sample

Set of observations used in an analysis.

IN USE document the sample

distribution

Pattern of observed values across a range.

IN USE inspect the latency distribution

outlier

Observation unusually distant from most other values.

IN USE investigate an outlier

measurement noise

Variation not attributable to the intended comparison alone.

IN USE account for measurement noise

controlled conditions

Specified conditions held consistent during comparison.

IN USE preserve controlled conditions

production behavior

Behavior under actual live operating conditions.

IN USE avoid assuming production behavior

bottleneck

Constraint limiting overall system performance.

IN USE identify the bottleneck

budget compliance

Whether measured usage satisfies the stated limit.

IN USE check budget compliance

Precision check

A uses 80 megabytes, leaving 40 below the 120-megabyte budget. B uses 160, exceeding it by 40. Neither median describes the unmeasured tail, and neither controlled result establishes production behavior.

Phrases you can actually use

C Notice the language / 1 of 2

NAME THE CONDITIONS

"Both runs use the same input and machine."

STATE B

"B has a twenty-five-millisecond median and uses one hundred sixty megabytes."

NAME THE BASELINE

"That is a thirty-seven-point-five percent reduction relative to A."

STATE THE CONSTRAINT

"The memory budget is one hundred twenty megabytes."

STATE A

"A has a forty-millisecond median and uses eighty megabytes."

REPORT THE ABSOLUTE GAIN

"B lowers the median by fifteen milliseconds."

REPORT THE COST

"B uses twice A's measured memory."

CALCULATE THE OVERRUN

"B exceeds that budget by forty megabytes."

06

Relative to A

Names the denominator for the percentage comparison.

By versus to

By gives the change; to gives the resulting value.

Twice

Compares B's memory with A's, not with the budget.

Phrases you can actually use

C Notice the language / 2 of 2

CALCULATE THE HEADROOM

"A remains forty megabytes below the limit."

LIMIT THE METRIC

"The median does not tell us the tail latency."

SEPARATE THROUGHPUT

"These figures do not establish throughput."

KEEP THE DECISION BOUNDED

"The measured gain does not remove the budget constraint."

AVOID A VAGUE VERDICT

"B is faster on this median measure, not automatically better overall."

NAME THE MISSING EVIDENCE

"We have not measured production behavior."

REQUEST A BALANCED SUMMARY

"Put the latency gain and memory cost together."

CLOSE WITHOUT OVERCLAIMING

"We have a documented tradeoff, not a production guarantee."

06

On this measure

Limits a faster claim to the reported metric.

Does not tell us

Prevents inference from median to unmeasured tail behavior.

Not automatically

Preserves the benefit while resisting an overall conclusion.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. What is B's median reduction relative to A?

- A 100 percent
- B 37.5 percent
- C 60 percent
- D 15 percent

2. Which budget statement is correct?

- A B is 80 below budget
- B A is 40 above budget
- C A is 40 below and B is 40 above
- D Both are within budget

3. Which claim exceeds the evidence?

- A B has a lower measured median
- B B uses more measured memory
- C The benchmark used the same machine
- D B improves every production request

4. Which summary is balanced?

- A B is universally superior
- B A is faster in the supplied test
- C Memory no longer matters because latency improved
- D B lowers median latency but exceeds the memory budget; tail and production remain unmeasured

Answers and explanations: [page 95](#).

Fifteen milliseconds faster, forty megabytes over budget

E Conversations / turns 1-10

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

06

WORD + PHRASE BANK

budget compliance / benchmark / tradeoff / Production behavior / headroom / Tail latency / controlled conditions / relative reduction / memory footprint / absolute difference

- 01 **Sora:** B's measured median is twenty-five milliseconds versus A's forty on the same input and machine. I'd like to propose B, but its memory figure complicates that.
- 02 **Malik:** That is useful ¹ _____ evidence. Put the memory figures beside it before we call B the better option. The resource budget is part of this decision.
- 03 **Sora:** A uses eighty megabytes; B uses a hundred sixty. The budget is a hundred twenty, so B is forty over even in this measured case.
- 04 **Malik:** Right. Its ² _____ is twice A's and forty megabytes over budget. The latency improvement does not cancel that constraint.
- 05 **Sora:** For the percentage, I subtracted twenty-five from forty and divided the fifteen-millisecond decrease by forty. That gives thirty-seven-point-five percent.
- 06 **Malik:** Yes, that is the ³ _____ relative to A. Keep the baseline explicit so nobody divides by twenty-five and reports a different quantity under the same label.
- 07 **Sora:** Would faster be too loose in the headline? I measured latency, not how many requests per second either version handles.
- 08 **Malik:** Prefer the precise ⁴ _____ and percentage latency reduction. Those statements tell readers exactly which measurement changed without suggesting an unmeasured throughput result.
- 09 **Sora:** We didn't collect tail percentiles. A lower median could coexist with worse slow requests, so I can't claim improvement for every request.
- 10 **Malik:** Correct. ⁵ _____ remains unmeasured. Do not turn a better middle value into a claim about every request or a specific percentile we did not record.

The conversation, continued

E Conversations / turns 11-20

Continue with the word bank on page 55.

- 11 **Sora:** A is forty megabytes below the same budget. I'll keep that separate from B's forty-megabyte excess rather than call both numbers spare memory.
- 12 **Malik:** That is forty megabytes of ⁶ _____ under the stated budget. It is a different claim from saying A has more memory available under all possible workloads.
- 13 **Sora:** These are benchmark runs, not live measurements. Should I put that boundary alongside the numbers instead of burying it in a note?
- 14 **Malik:** Yes. ⁷ _____ is not established by these figures. Name that limit instead of relying on readers to infer it from the word benchmark.
- 15 **Sora:** Then the proposal needs both sides: fifteen milliseconds lower median, twice the memory, and B above the present limit.
- 16 **Malik:** That presents the ⁸ _____ fairly. It acknowledges a real improvement without treating one favorable number as a complete engineering decision.
- 17 **Sora:** I could request a changed memory budget, but that wouldn't make this run compliant with the current one. It would be a separate decision.
- 18 **Malik:** Exactly. Current ⁹ _____ is clear: A is below the limit and B is above it. A hypothetical future budget should not rewrite today's comparison.
- 19 **Sora:** I'll report the matched conditions, latency decrease, memory figures, and unmeasured tail and live behavior. We still need to decide what to investigate next.
- 20 **Malik:** Good. Keep the ¹⁰ _____ visible alongside that summary. We can make a reasoned next decision without pretending the benchmark answers questions it never measured.

06

ANSWERS / EXPLANATIONS: PAGE 96

1. benchmark | 2. memory footprint | 3. relative reduction | 4. absolute difference | 5. Tail latency | 6. headroom | 7. Production behavior | 8. tradeoff | 9. budget compliance | 10. controlled conditions

Report both sides of the comparison

F Say it

Use the supplied numbers to complete the benchmark summary. Units are printed in each sentence.

Complete the four gaps. Use each bank entry once.

06

WORD + PHRASE BANK

160 / 25 / 15 / 40

Developer: "B's median is ¹ _____ milliseconds."

Reviewer: "The median decreases by ² _____ milliseconds relative to A."

Developer: "B uses ³ _____ megabytes of memory."

Reviewer: "B exceeds the budget by ⁴ _____ megabytes."

ANSWERS / EXPLANATIONS: PAGE 96

1. 25 | 2. 15 | 3. 160 | 4. 40

Rehearse with a fixed script

1. Read the performance comparison aloud with units: milliseconds for latency, megabytes for memory.
2. Swap roles for turns 5-12. State the fifteen-millisecond decrease, 37.5 percent reduction, and forty-megabyte headroom accurately.
3. Compare the final report with the supplied table facts. Keep the budget violation and unmeasured production behavior visible.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

The timeout did not answer the business question

A Read the situation

Explain an uncertain request outcome and request a retry contract before risking a second reservation.

A fictional reservation client times out before receiving a reply. The service may already have created the reservation. A proposed automatic retry would submit a new reservation request, and the service's duplicate-handling behavior is unspecified. Developers Imani and Leo discuss the risk. No production change is authorized. They need to clarify outcome reconciliation and retry semantics, then test agreed behavior in a suitable controlled environment.

PEOPLE / Imani: Client developer | Leo: Service developer

07

DISTINGUISH NOT KNOWING FROM KNOWING FAILURE

Timeout language can silently collapse uncertainty into failure. Say what the client observed and what the service may have done. This makes a request for an explicit retry contract sound like necessary coordination rather than unnecessary caution.

Briefing checks

Circle one answer in each item. Use the briefing.

1. What did the client observe?

- A A verified cancellation
- C Confirmed rejection before processing

- B No reply before its timeout
- D Confirmed creation twice

2. What might the service have done?

- A Created the reservation already
- C Guaranteed cancellation

- B Definitely done nothing
- D Automatically merged duplicates

3. What is unspecified?

- A Whether the client timed out
- C Whether production authorization exists

- B Whether the retry is proposed as new
- D Duplicate-handling behavior for the proposed retry

Answers and explanations: [page 97](#).

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

timeout

End of a waiting period without the required response.

IN USE observe a timeout

retry

Repeated attempt after an earlier uncertain or unsuccessful attempt.

IN USE define retry behavior

request identity

Identifier or meaning used to distinguish one operation from another.

IN USE preserve request identity

idempotency

Property allowing repeated equivalent requests without additional intended effects.

IN USE specify idempotency semantics

idempotency key

Identifier used under an agreed contract to recognize repeated intent.

IN USE define an idempotency key

duplicate request

Repeated request that may represent the same intended operation.

IN USE detect a duplicate request

duplicate side effect

Additional effect caused by repeating an operation.

IN USE prevent a duplicate side effect

shared state

Data or condition used by multiple participants or operations.

IN USE reconcile shared state

uncertain outcome

Situation where the operation's final result is not known.

IN USE report an uncertain outcome

acknowledgment

Response indicating receipt or completion as defined by the protocol.

IN USE distinguish an acknowledgment

reconciliation

Checking records to establish and align the actual outcome.

IN USE request outcome reconciliation

at-least-once delivery

Delivery model allowing one or more deliveries of a message.

IN USE discuss at-least-once delivery

Precision check

A timeout describes the client's wait, not proof that the service did nothing. Submitting a new reservation could create another business effect if the first request already succeeded and duplicate handling is not defined.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

at-most-once delivery

Delivery model allowing no more than one delivery, with possible loss.

IN USE distinguish at-most-once delivery

deduplication

Recognition and handling of repeated requests or records.

IN USE define deduplication behavior

race condition

Behavior depending on timing among concurrent operations.

IN USE investigate a race condition

concurrency

Overlapping progress of multiple operations.

IN USE account for concurrency

atomic operation

Operation treated as an indivisible state change under its model.

IN USE define the atomic operation

transaction boundary

Scope within which specified changes are coordinated.

IN USE clarify the transaction boundary

backoff

Delay strategy between repeated attempts.

IN USE configure retry backoff

retry budget

Limit on permitted repeated attempts under a policy.

IN USE set a retry budget

correlation identifier

Identifier linking related events across components.

IN USE retain the correlation identifier

status lookup

Query intended to retrieve an operation's recorded state.

IN USE define a status lookup

sandbox

Controlled environment separated from live operations.

IN USE test in a sandbox

authorization

Permission for a specified action or change.

IN USE obtain production authorization

Precision check

An idempotency key is not a magic guarantee. Its scope, reuse rules, retention, responses, and service behavior need a contract. Backoff changes timing; it does not by itself resolve an uncertain outcome or prevent duplication.

Phrases you can actually use

C Notice the language / 1 of 2

STATE THE OBSERVATION

"The client timed out before receiving a reply."

AVOID A FALSE CONCLUSION

"No reply does not prove no reservation."

NAME THE RISK

"That could create a duplicate business effect."

AVOID ASSUMING SUPPORT

"Duplicate handling is not specified."

PRESERVE UNCERTAINTY

"The service may already have created the reservation."

IDENTIFY THE PROPOSAL

"The retry would submit a new reservation request."

ASK FOR THE CONTRACT

"How does the service recognize repeated intent?"

CLARIFY IDENTITY

"Would this carry the same request identity or a new one?"

May already have

Expresses a possible completed action whose result is not yet known.

Does not prove

Blocks an invalid inference from silence to failure.

Could create

Names a risk without falsely asserting duplication already occurred.

Phrases you can actually use

C Notice the language / 2 of 2

REQUEST RECONCILIATION

"Can we establish the outcome of the first request?"

QUALIFY THE KEY

"An idempotency key needs agreed service behavior."

REQUEST A CONTROLLED CHECK

"Test the agreed behavior in a suitable sandbox."

KEEP THE DESIGN PROVISIONAL

"Automatic retry remains a proposal."

SEPARATE TIMING AND SEMANTICS

"Backoff does not itself prevent duplicates."

STATE THE PERMISSION BOUNDARY

"No production change is authorized."

NAME THE MISSING EVIDENCE

"We do not know whether the first reservation exists."

CLOSE WITH THE DEPENDENCY

"Define the contract before claiming retry safety."

Same ... or new

Exposes request identity as a design decision.

By itself

Limits what backoff or a key can guarantee alone.

Before claiming

Makes a confidence statement dependent on missing contract evidence.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. Which statement follows from the timeout?

- A** Two reservations definitely exist
- B** The service canceled the request
- C** The client did not receive a timely reply
- D** No reservation exists

2. Why is a new reservation request risky?

- A** Backoff guarantees a second failure
- B** The first may already have created a reservation
- C** Every retry always duplicates
- D** The client has confirmed cancellation

3. What does an idempotency key require?

- A** No server participation
- B** Automatic universal guarantees
- C** Agreed service semantics
- D** Only a convenient label

4. Which next step is appropriate?

- A** Enable the retry in production immediately
- B** Assume timeout means rejection
- C** Promise exactly one reservation without evidence
- D** Clarify reconciliation and retry behavior, then test it outside production

[Answers and explanations: page 97.](#)

The timeout did not answer the business question

E Conversations / turns 1-10

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

WORD + PHRASE BANK

shared state / timeout / deduplication / Backoff / request identity / reconciliation / uncertain outcome / status lookup / idempotency key / authorization

07

- 01 **Imani:** The reservation client stopped waiting before a reply arrived. My retry proposal creates a new request, and I'm worried the first reservation may already exist.
- 02 **Leo:** We need to preserve the ¹ _____ of the first attempt. The service may already have created it, so the timeout is not proof that nothing happened.
- 03 **Imani:** I wrote request failed in the incident note. That's too strong: the only confirmed fact is that the client didn't get a response in time.
- 04 **Leo:** Exactly. A ² _____ is an observation at the client. We need separate evidence about the service state before turning that into a business conclusion.
- 05 **Imani:** If the service completed the first request, our new request could create another reservation. We don't have a documented rule that combines them.
- 06 **Leo:** Then ³ _____ must remain unspecified in the proposal. We cannot promise the second request will be merged or ignored just because the input looks similar.
- 07 **Imani:** Could we use a key to repeat the original intent? I don't want to add a field and assume the service knows how to handle it.
- 08 **Leo:** An ⁴ _____ needs agreed semantics on the service side. A new field alone does not tell us its scope, reuse rules, or what response a repeat receives.
- 09 **Imani:** Then we need to specify whether this is another attempt at one booking or a genuinely new booking. The identifiers must reflect that distinction.
- 10 **Leo:** Yes. Define the ⁵ _____ and how the service interprets it. That distinction belongs in the contract, not in an assumption hidden inside the client's retry loop.

The conversation, continued

E Conversations / turns 11-20

Continue with the word bank on page 64.

- 11 **Imani:** Is there a supported way to check the first outcome? I can ask about a status operation, but I haven't established that one exists.
- 12 **Leo:** Request an agreed ⁶ _____ or another supported reconciliation process. Its availability and meaning need confirmation; we have not established either in this discussion.
- 13 **Imani:** Someone suggested waiting longer before retrying. That might reduce traffic, but wouldn't it leave the duplicate-reservation question exactly where it is?
- 14 **Leo:** Correct. ⁷ _____ is a timing policy, not a substitute for outcome and duplicate-handling semantics. A slower duplicate is still a possible duplicate.
- 15 **Imani:** I'll keep automatic retry disabled while those rules are unresolved. This investigation isn't permission to change live reservation behavior.
- 16 **Leo:** Keep that ⁸ _____ boundary visible. Once the contract is defined, we can plan a controlled check without changing live reservation behavior.
- 17 **Imani:** For the controlled test, I want the service to complete while its reply is delayed. Testing only an immediate rejection misses the uncertain-outcome case.
- 18 **Leo:** That will address the relevant ⁹ _____ uncertainty. Use a suitable sandbox and define the expected outcome before interpreting the test as evidence.
- 19 **Imani:** My revised note will say no timely reply, first outcome unknown, retry currently new, duplicates unspecified, and no production change approved.
- 20 **Leo:** Good. We need ¹⁰ _____ and an explicit retry contract before claiming safety. That is the concrete next discussion, not a guarantee that an extra attempt will be harmless.

07

ANSWERS / EXPLANATIONS: PAGE 98

1. uncertain outcome | 2. timeout | 3. deduplication | 4. idempotency key | 5. request identity | 6. status lookup | 7. Backoff | 8. authorization | 9. shared state | 10. reconciliation

Do not turn silence into failure

F Say it

Complete the retry discussion using the stated uncertainty and permission boundary.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

authorized / timeout / unspecified / exist

07

Developer: "The client observed a ¹ _____."

Reviewer: "The first reservation may already ² _____."

Developer: "Duplicate handling remains ³ _____."

Reviewer: "No production change is ⁴ _____."

ANSWERS / EXPLANATIONS: PAGE 98

1. timeout | 2. exist | 3. unspecified | 4. authorized

Rehearse with a fixed script

1. Read the timeout report without replacing no reply with confirmed failure.
2. Swap roles and repeat turns 5-14. Distinguish request identity, outcome lookup, and retry timing.
3. Repeat the final four turns. The first outcome remains unknown and production retry remains unauthorized.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

R18 is deployed, but the release decision is not made

A Read the situation

Hand over environment, flag state, migration limits, and decision ownership without conflating deployment with production readiness.

Release R18 is deployed in staging. Its new search feature is disabled by a feature flag. A migration adds an optional field, but compatibility with the previous application version has not been tested. Developer Chen hands the status to release reviewer Lee. The production decision remains with Lee and has not been made. Neither the optional field nor the staging deployment establishes that an older application will safely work with the migrated data.

PEOPLE / Chen: Developer handing over R18 | Lee: Release reviewer

SAY DONE WITH AN OBJECT ATTACHED

Deployed, enabled, tested, and approved describe different milestones. Attach each status to an environment, feature, or specific check. This reduces pressure on the next person to infer readiness from a single broad word such as ready or complete.

Briefing checks

Circle one answer in each item. Use the briefing.

1. Where is R18 deployed?

- A Production
- C Nowhere

- B Every customer environment
- D Staging

2. What is the new search feature's state?

- A Enabled for all users
- C Proven compatible with every client

- B Deleted from the release
- D Disabled by a feature flag

3. Which check is unresolved?

- A Compatibility with the previous application version
- C Whether the field is optional

- B Whether the staging deployment occurred
- D Who owns the production review

Answers and explanations: page 99.

The vocabulary that does the work

B Find the words / 1 of 2

Read the definitions and say the common word combinations aloud.

release candidate

Version proposed for release pending the relevant checks.

IN USE review the release candidate

deployment

Placement of software into a specified environment.

IN USE report the deployment

staging

Preproduction environment used for relevant checks.

IN USE deploy to staging

production

Live environment serving actual operational users or workloads.

IN USE authorize production release

feature flag

Control that enables or disables specified behavior.

IN USE verify the feature flag

disabled state

Condition in which a feature is not enabled.

IN USE preserve the disabled state

rollout

Process of making a change available to intended users.

IN USE plan the rollout

migration

Change to stored data or its structure.

IN USE review the migration

optional field

Field that may be absent under the applicable rules.

IN USE add an optional field

schema compatibility

Ability of components to work with the relevant data structure.

IN USE verify schema compatibility

previous version

Earlier application version relevant to the comparison.

IN USE test the previous version

rollback

Return to an earlier application or system state under a defined plan.

IN USE assess rollback readiness

Precision check

R18 is deployed in staging, and search is disabled. Those are two separate facts. They do not imply production deployment, enabled search, completed compatibility testing, or Lee's approval to proceed.

The vocabulary that does the work

B Find the words / 2 of 2

Read the definitions and say the common word combinations aloud.

forward compatibility

Ability to work with relevant newer data or interfaces.

IN USE check forward compatibility

backward compatibility

Ability to preserve supported use by earlier consumers.

IN USE verify backward compatibility

release gate

Required condition before advancing a release.

IN USE satisfy the release gate

sign-off

Explicit approval at a stated decision point.

IN USE obtain release sign-off

go/no-go decision

Decision whether to proceed under the relevant release process.

IN USE record the go/no-go decision

handoff

Transfer of relevant status, evidence, and responsibility.

IN USE prepare the release handoff

known limitation

Identified restriction or unresolved boundary.

IN USE list known limitations

verification matrix

Organized record of combinations and checks to be verified.

IN USE update the verification matrix

environment boundary

Limit separating claims about different execution settings.

IN USE preserve the environment boundary

data compatibility

Ability to interpret and use the relevant stored data correctly.

IN USE test data compatibility

release owner

Person responsible for the specified release decision.

IN USE name the release owner

readiness claim

Statement that required conditions for proceeding are met.

IN USE qualify the readiness claim

Precision check

An optional new field may look low risk, but its optional status does not prove an older application can handle the changed data or schema. Rollback readiness requires evidence about the earlier version and the actual post-migration state.

Phrases you can actually use

C Notice the language / 1 of 2

IDENTIFY THE RELEASE

"This handoff concerns R18."

NAME THE FEATURE STATE

"New search remains disabled by its flag."

DESCRIBE THE DATA CHANGE

"The migration adds an optional field."

AVOID A SHORTCUT

"Optional does not prove compatibility."

NAME THE ENVIRONMENT

"R18 is deployed in staging."

SEPARATE DEPLOYMENT AND USE

"Deployed does not mean enabled."

STATE THE MISSING CHECK

"Compatibility with the previous application version is untested."

NAME THE DECISION OWNER

"Lee owns the production review."

08

Remains disabled

Preserves feature state despite completed deployment.

Against

Connects the previous application to the data state it must handle.

Not production sign-off

Prevents a handoff from being mistaken for permission.

Phrases you can actually use

C Notice the language / 2 of 2

KEEP THE DECISION OPEN

"The production decision has not been made."

REQUEST THE EXACT EVIDENCE

"Check the previous application against the migrated state."

IDENTIFY THE PENDING ITEM

"The compatibility check remains open."

PRESERVE THE BOUNDARY

"Do not describe search as live."

LIMIT THE READINESS CLAIM

"This is a staging status, not production sign-off."

SEPARATE ROLLBACK ASSUMPTIONS

"An older build alone does not establish rollback readiness."

ASK FOR A TRACEABLE DECISION

"Record the review outcome explicitly."

CLOSE WITH STATUS

"Staging deployed, search disabled, compatibility pending, decision with Lee."

08

Has not been made

Makes the unresolved decision explicit.

Alone

Limits what an available earlier build establishes.

With Lee

Assigns decision ownership without predicting approval.

Choose the precise message

D Check your understanding

Circle one best answer for each item. Use the stated purpose and case facts, not the option that merely sounds most polite.

1. Which status is accurate?

- A** R18 has full production sign-off
- B** The old application is verified compatible
- C** R18 is in staging with search disabled
- D** Search is live in production

2. Why does optional not settle compatibility?

- A** Optional fields are always incompatible
- B** The migration never ran anywhere
- C** Lee has already rejected release
- D** The previous application's behavior with migrated data is untested

3. Which rollback claim is justified?

- A** An older build guarantees safe rollback
- B** Staging deployment proves data reversibility
- C** A disabled flag automatically undoes the migration
- D** Readiness remains unverified without the relevant compatibility evidence

4. Who owns the unresolved production decision?

- A** The feature flag itself
- B** The migration tool
- C** Lee
- D** Every staging user

Answers and explanations: [page 99](#).

R18 is deployed, but the release decision is not made

E Conversations / turns 1-10

Complete the numbered gaps. Use each bank entry once, without changing its form. Cover the answers at the end until you finish.

WORD + PHRASE BANK

rollback / readiness claim / environment boundary / feature flag / Schema compatibility / data compatibility / release gate / handoff / known limitation / go/no-go decision

08

- 01 **Chen:** R18 is deployed in staging. Search is still disabled, and I need your release decision after we address the compatibility gap.
- 02 **Lee:** Keep that ¹ _____ in the first sentence. Someone reading only the headline mustn't mistake staging deployment for a production launch.
- 03 **Chen:** I'll put the flag status next to the environment. The code being present doesn't mean users can use the new search.
- 04 **Lee:** Right. The ² _____ remains off. Installed code, enabled behavior, and production approval are separate facts.
- 05 **Chen:** The migration adds an optional field. I called it backward compatible in the draft, but we haven't tested the prior application against the migrated data.
- 06 **Lee:** Make that a ³ _____ instead. Optionality is a property of the field, not a result from running the previous application.
- 07 **Chen:** So I'll replace the claim with prior-version compatibility untested. I don't have evidence that the older reader tolerates the changed data.
- 08 **Lee:** Correct. ⁴ _____ needs the relevant combination checked. Successful deployment of the new build doesn't answer how the old build handles it.
- 09 **Chen:** We do have the old build available. Is that enough to say we can revert if the release causes a problem?
- 10 **Lee:** No. ⁵ _____ readiness depends on more than having the executable. The data state after migration matters to that path.

The conversation, continued

E Conversations / turns 11-20

Continue with the word bank on page 73.

- 11 **Chen:** Then the open check is the previous application with the migrated state, not simply confirming that its build artifact can be downloaded.
- 12 **Lee:** Exactly. Request ⁶ _____ evidence for that combination. Don't substitute an artifact-availability check for an operational result.
- 13 **Chen:** I'll leave production pending. Sending you the handoff doesn't authorize me to turn search on or change the production environment.
- 14 **Lee:** Correct. I own the ⁷ _____, and it hasn't been made. Please don't phrase my review as approval that's already expected.
- 15 **Chen:** The handoff will list R18, staging, search disabled, the optional-field change, and the missing prior-version check with you as decision owner.
- 16 **Lee:** That's a usable ⁸ _____. Concrete states tell me what's complete and what I still need; ready on its own doesn't.
- 17 **Chen:** When we get the compatibility result, I'll attach the actual evidence. Until then the successful staging step stays separate from that open check.
- 18 **Lee:** Good. A ⁹ _____ isn't met simply because another step completed. We'll review the missing evidence before deciding whether to advance.
- 19 **Chen:** Final status: staging deployed, search disabled, compatibility pending, production decision with Lee. No statement that search is live.
- 20 **Lee:** That keeps the ¹⁰ _____ defensible. Now the next person can act on the unresolved check without having to undo an implied release approval.

08

ANSWERS / EXPLANATIONS: PAGE 100

1. environment boundary | 2. feature flag | 3. known limitation | 4. Schema compatibility | 5. rollback | 6. data compatibility | 7. go/no-go decision | 8. handoff | 9. release gate | 10. readiness claim

Four facts for the release handoff

F Say it

Complete the R18 handoff. Do not upgrade any pending check or decision to completed status.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

Lee / untested / staging / disabled

08

Developer: "R18 is deployed in ¹ _____."

Reviewer: "New search remains ² _____."

Developer: "Prior-version compatibility is ³ _____."

Reviewer: "The production decision remains with ⁴ _____."

ANSWERS / EXPLANATIONS: PAGE 100

1. staging | 2. disabled | 3. untested | 4. Lee

Rehearse with a fixed script

1. Read the handoff with a pause after each status: environment, flag, compatibility, decision.
2. Swap roles for turns 5-12. Stress untested prior-version behavior rather than assuming optional means compatible.
3. Check the final readback against turns 19-20. Staging deployment must not become a claim of a live feature or production approval.

WORKING ALONE

Read both roles, pausing for two seconds when the speaker changes. Repeat with the bank covered, then check any missed expressions.

The newer tag cannot repair an older payload

E More workplace conversations / 1-10

Explain a rejected conditional update without silently overwriting another person's change.

Fictional whole-record PUT replaces price and description. Both clients read \$18, "old", strong ETag "v7". A changes price to \$20, creating strong tag "v8". B submits \$18, "new", If-Match: "v7". B's change has not previously succeeded. This service returns 412 without changing data when the condition fails. No later update is approved.

PEOPLE / Dina: Client developer | Marco: Service developer

Use each bank entry once, without changing its form. Answers follow turn 20.

09

WORD + PHRASE BANK

strong ETag / conflict reconciliation / stale representation / lost update / If-Match precondition / HTTP 412

- 01 **Dina:** B got a 412 saving the description. Its payload still contains eighteen dollars, because both clients started from the same version.
- 02 **Marco:** That is the ¹ _____ we need to discuss. A has already changed the price to twenty; B hasn't seen that change.
- 03 **Dina:** The request includes v7. Is the server comparing that with the current version, rather than comparing our description text?
- 04 **Marco:** Yes. The ² _____ fails here. The response is 412 with no update, not evidence that B's description was saved.
- 05 **Dina:** Could I fetch just the new tag and resubmit exactly the same body? That would get past the version check.
- 06 **Marco:** It could cause a ³ _____. The body still carries eighteen. Passing a version check wouldn't make that overwrite intentional.
- 07 **Dina:** So we need the current record as well as its tag. We should show the price change before preparing another full replacement.
- 08 **Marco:** Exactly. A ⁴ _____ identifies the representation you're checking against. It does not tell us which conflicting fields the user meant to replace.
- 09 **Dina:** The status text says Precondition Failed. Someone called it a server outage, but the service is rejecting our condition as designed.
- 10 **Marco:** Keep the ⁵ _____ meaning precise. The price remains twenty and the description remains old after this rejected request.

The conversation, continued

E More workplace conversations / 11-20

Continue with the word bank on page 76.

- 11 **Dina:** For the next attempt, we'll retrieve the current record, review the difference, and preserve twenty if the user only wants to change the description.
- 12 **Marco:** That's ⁶ _____, not a blind retry. Another update would still need the current condition checked when it reaches the service.
- 13 **Dina:** Meaning someone else could change the record between our fresh read and our next submission. Fetching once doesn't lock it.
- 14 **Marco:** Right. Optimistic concurrency detects a changed version at the conditional write. It doesn't reserve the record while the user is reading it.
- 15 **Dina:** Our error message should preserve the user's unsaved description rather than discard it when we fetch the newer record.
- 16 **Marco:** Agreed as a proposed interface behavior. Show the fresh record and pending edit separately so the user can review what will actually be sent.
- 17 **Dina:** For the test, I'll interleave two clients: A saves twenty first, then B sends the stale condition and gets the specified rejection.
- 18 **Marco:** Check the stored fields afterward too. A status assertion alone wouldn't catch a defective implementation that rejects the request but still changes the data.
- 19 **Dina:** Then the handoff is: B's save rejected, A's twenty preserved, description unchanged, and a reviewed resubmission still pending.
- 20 **Marco:** Yes. Don't label it resolved merely because we know v8. The condition protects the write; the client still has to prepare the right change.

09

ANSWERS / EXPLANATIONS: PAGE 101

1. stale representation | 2. If-Match precondition | 3. lost update | 4. strong ETag | 5. HTTP 412 | 6. conflict reconciliation

A rejected edit is not a merged edit

F Say it in a new situation

Same fictional service. Current strong tag "v12" has quantity 9 and note "old". A stale PUT sends quantity 7, note "new", If-Match: "v11". It has not previously succeeded. The service returns 412 without changing data. No resubmission occurs.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

stale / 9 / old / 412

09

Developer: The returned status is ¹ _____.

Reviewer: The stored quantity remains ² _____.

Developer: The stored note is still ³ _____.

Reviewer: Replacing only the tag would leave the payload ⁴ _____.

ANSWERS / EXPLANATIONS: PAGE 101

1. 412 | 2. 9 | 3. old | 4. stale

Read, switch, repeat

1. Check the four answers and read the complete exchange aloud.
2. Switch roles. Read it again with the word bank covered.
3. Return to the extended conversation. Read turns 1-10, then switch roles for turns 11-20. Use the corrected expressions.

SOLO PRACTICE

Read both roles. Pause briefly at each change of speaker. Repeat any line that needed an answer check.

Fifty orders, fifty-one queries

E More workplace conversations / 1-10

Explain query-count growth and a batching proposal without promising an unmeasured response time.

Fictional order page: one query loads 50 orders, then one separate query per order loads its lines. All 51 calls are serial; no cache or other queries. A proposed batch implementation uses one order query and one line query for those 50 IDs. For this simplified model, each database round trip contributes exactly 4 ms of network delay. Database work, data transfer, and application work are additional and unmeasured.

PEOPLE / Jules: Application developer | Amir: Database specialist

Use each bank entry once, without changing its form. Answers follow turn 20.

10

WORD + PHRASE BANK

batched lookup / round-trip count / result set / N+1 query pattern / network component / lazy loading

- 01 **Jules:** The page has fifty orders, but the trace shows fifty-one queries. The first fetch looks normal; the extra calls appear while we render the lines.
- 02 **Amir:** That matches the ¹ _____ in this trace. Loading the orders once doesn't mean their related lines were loaded with them.
- 03 **Jules:** Our object-relational mapper fetches the lines when the template first accesses each collection. I hadn't noticed that property access was issuing SQL.
- 04 **Amir:** That's ² _____ here. It isn't inherently wrong, but this access pattern produces one database call per order.
- 05 **Jules:** The proposed version fetches the orders, collects their IDs, then fetches all matching lines together. That would be two queries for this page.
- 06 **Amir:** Yes, the proposed ³ _____ reduces this example from fifty-one to two. Check that each line is attached to its correct order afterward.
- 07 **Jules:** At four milliseconds per serial round trip, the old path has two hundred four milliseconds of network delay before we add other work.
- 08 **Amir:** And the new ⁴ _____ contributes eight milliseconds under that model. That's a hundred ninety-six milliseconds less modeled network delay.
- 09 **Jules:** I'll put eight milliseconds in the response-time target, then. The arithmetic makes it look very fast.
- 10 **Amir:** Call it the ⁵ _____, not the whole response. We haven't timed query execution, transferred rows, or object construction.

The conversation, continued

E More workplace conversations / 11-20

Continue with the word bank on page 79.

- 11 **Jules:** A larger batch could load more rows at once. Fewer queries don't automatically mean less memory or less database work.
- 12 **Amir:** Exactly. Measure the ⁶ _____ as well. An order with no lines must still appear if the page contract includes it.
- 13 **Jules:** I'll compare the rendered orders and line associations, not only the query counter. We could get two queries and still show the wrong data.
- 14 **Amir:** Good. Keep ordering and empty collections in that check. The optimization must preserve the page's specified result.
- 15 **Jules:** For a bigger page, is one giant query always the best answer? Some database parameter limits would affect a long list of IDs.
- 16 **Amir:** The batch size needs an implementation decision. Our fifty-ID example isn't a universal limit or a promise that all frameworks use that size.
- 17 **Jules:** Then I'll label fifty as this case's input, with two queries for the proposed version. We still need measurements on representative data.
- 18 **Amir:** Yes. Record query count, total time, and relevant resource use separately. A changed query count is strong evidence about calls, not everything else.
- 19 **Jules:** The review summary will say fifty-one versus two queries, modeled network delay two hundred four versus eight milliseconds, actual total latency not yet measured.
- 20 **Amir:** That is a defensible comparison. It explains the mechanism and arithmetic without turning a simplified calculation into a production guarantee.

10

ANSWERS / EXPLANATIONS: PAGE 102

1. N+1 query pattern | 2. lazy loading | 3. batched lookup | 4. round-trip count | 5. network component | 6. result set

Batch size changes the query count

F Say it in a new situation

New fictional page: 200 orders, one initial order query, no cache. Old path makes one additional line query per order. New implementation explicitly groups IDs into batches of 50, one line query per batch. All calls are serial at 4 ms network delay each; other work remains unmeasured.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

20 / 201 / 784 / 5

10

Developer: The batched path makes ¹ _____ total queries.

Reviewer: Its modeled network delay is ² _____ milliseconds.

Developer: The old path makes ³ _____ total queries.

Reviewer: The modeled network-delay reduction is ⁴ _____ milliseconds.

ANSWERS / EXPLANATIONS: PAGE 102

1. 5 | 2. 20 | 3. 201 | 4. 784

Read, switch, repeat

1. Check the four answers and read the complete exchange aloud.
2. Switch roles. Read it again with the word bank covered.
3. Return to the extended conversation. Read turns 1-10, then switch roles for turns 11-20. Use the corrected expressions.

SOLO PRACTICE

Read both roles. Pause briefly at each change of speaker. Repeat any line that needed an answer check.

One cent, two different contracts

E More workplace conversations / 1-10

Explain why rounding stage and rounding mode belong in an executable money calculation contract.

Fictional checkout test, not tax advice. Three separate line items cost \$0.05 each before a fictional 10% charge. The agreed contract calculates the charge on each line, rounds it to two decimal places using half-up, then sums the rounded charges. No discounts or other charges. A proposed implementation instead calculates 10% of the basket subtotal and rounds only that result. Use exact decimal inputs.

PEOPLE / Rina: Checkout developer | Sol: Reviewer

Use each bank entry once, without changing its form. Answers follow turn 20.

WORD + PHRASE BANK

basket-level calculation / decimal arithmetic / rounding stage / half-up rule / rounding mode / line-item boundary

11

- 01 **Rina:** My checkout total is seventeen cents, but the acceptance example says eighteen. Both calculations use ten percent, so I thought the example had a typo.
- 02 **Sol:** Check the ¹ _____. The contract rounds each line's charge before adding the charges together.
- 03 **Rina:** Each five-cent line produces half a cent before rounding. With this rule, that becomes one cent per line, three cents in all.
- 04 **Sol:** Yes. The specified ² _____ sends 0.005 to 0.01. The fifteen-cent subtotal plus three cents gives eighteen.
- 05 **Rina:** My version applies ten percent to fifteen cents, giving one and a half cents, then rounds to two cents. That explains seventeen.
- 06 **Sol:** That's a ³ _____. Its arithmetic isn't the agreed algorithm. Equal rates don't make the rounding stages interchangeable.
- 07 **Rina:** Would changing the display to two decimal places fix it? The customer would still see a neatly formatted amount.
- 08 **Sol:** Formatting doesn't establish ⁴ _____. We need the calculation and its rounding decisions to match the contract first.
- 09 **Rina:** I'll construct the decimal values from the original decimal strings. Converting a binary floating-point approximation afterward can preserve the wrong intermediate value.
- 10 **Sol:** And make the ⁵ _____ explicit at the required operation. Don't assume the library default is half-up.

The conversation, continued

E More workplace conversations / 11-20

Continue with the word bank on page 82.

- 11 **Rina:** So the regression example must retain three separate five-cent lines. Combining them into one fifteen-cent line changes the meaning of the fixture.
- 12 **Sol:** Exactly. Preserve the ⁶ _____. The test should catch a future optimization that sums first and rounds later.
- 13 **Rina:** Could we store prices in integer cents instead? That would avoid binary approximations for the original prices.
- 14 **Sol:** It can represent those prices exactly, but the calculated charge still has fractional cents. Integer storage doesn't choose when or how to round them.
- 15 **Rina:** I also shouldn't throw away the half cent before applying the specified rounding. That would make each charge zero in this example.
- 16 **Sol:** Correct. Keep sufficient precision for the rule, then round at the agreed point. Truncating intermediate values would implement another algorithm.
- 17 **Rina:** I'll show the per-line charge, summed charge, subtotal, and total in the test evidence. Then the one-cent difference has a visible source.
- 18 **Sol:** Good. This fictional contract doesn't establish what any jurisdiction requires. A real checkout's approved financial rules must supply that decision.
- 19 **Rina:** The corrected expectation is three one-cent charges and an eighteen-cent total. My seventeen-cent result belongs to the rejected basket-level proposal.
- 20 **Sol:** That's the review conclusion. Fix the algorithm and verify this distinguishing example; don't just change the expected value to whatever the implementation currently returns.

11

ANSWERS / EXPLANATIONS: PAGE 103

1. rounding stage | 2. half-up rule | 3. basket-level calculation | 4. decimal arithmetic | 5. rounding mode | 6. line-item boundary

Preserve four separate lines

F Say it in a new situation

Same fictional line-level contract and exact decimal inputs, now four separate \$0.25 lines. Charge rate 10%; round each charge half-up to two decimal places, then sum. No other amounts. Compare with rounding 10% of the \$1.00 subtotal only once.

Complete the four gaps. Use each bank entry once.

WORD + PHRASE BANK

0.02 / 0.12 / 0.03 / 1.12

Developer: Each rounded line charge is ¹ _____ dollars.

Reviewer: The sum of the four rounded charges is ² _____ dollars.

Developer: The contract's final total is ³ _____ dollars.

Reviewer: That exceeds the basket-level total by ⁴ _____ dollars.

11

ANSWERS / EXPLANATIONS: PAGE 103

1. 0.03 | 2. 0.12 | 3. 1.12 | 4. 0.02

Read, switch, repeat

1. Check the four answers and read the complete exchange aloud.
2. Switch roles. Read it again with the word bank covered.
3. Return to the extended conversation. Read turns 1-10, then switch roles for turns 11-20. Use the corrected expressions.

SOLO PRACTICE

Read both roles. Pause briefly at each change of speaker. Repeat any line that needed an answer check.

01 Briefing and language checks

Clarifying interface contracts

A Briefing checks / page 4

1. What does omitting nickname do under the current contract?

B. Leaves the stored value unchanged

The current contract explicitly preserves the value when nickname is omitted.

2. What does an empty string do?

D. Clears the nickname

The empty string has a defined clearing meaning distinct from omission.

3. What remains undefined?

A. The meaning of an explicit null

The proposed client sends null, whose meaning the contract has not specified.

D Language checks / page 9

1. Which comment identifies the real gap?

C. The null case needs an agreed meaning

Only null lacks a specified meaning in this particular contract.

2. Which pair already has different defined effects in this contract?

B. Omission and an empty string

Omission preserves the value, whereas an empty string clears it.

3. What would justify calling null a clearing command?

D. An explicit agreed contract change

A new interpretation needs agreement rather than an unsupported inference.

4. Which test plan matches the discussion?

A. Check each defined input state and resolve null before asserting its effect

Distinct input states require distinct expectations, with null still unresolved.

01 Conversation answers

Clarifying interface contracts

E Dialogue gaps

Pages 10-11

1. omitted field. Turn 2. An omitted field is absent and preserves the stored nickname under the stated contract.

2. optional field. Turn 4. Optional field describes allowed absence, not the meaning or acceptability of an explicit null.

3. empty string. Turn 6. An empty string has the specified effect of clearing the nickname, unlike omission.

4. interface contract. Turn 8. The interface contract is the agreement that must specify the missing null behavior before implementation.

5. request payload. Turn 10. Request payload identifies the actual submitted data, which may differ from an assumed client representation.

6. field presence. Turn 12. Field presence distinguishes an absent nickname property from a present property carrying a value.

7. validation rule. Turn 14. The validation rule must reflect an agreed input policy rather than preempt the unresolved decision.

8. contract test. Turn 16. A contract test checks the agreed interaction across the client-service boundary rather than only the visible form.

9. backward compatibility. Turn 18. Backward compatibility concerns preserving supported behavior for existing clients, including the established omission rule.

10. acceptance criterion. Turn 20. An acceptance criterion must name the agreed input and effect before implementation can be judged complete.

F Transfer conversation

Page 12

1. unchanged

Developer: "An omitted nickname leaves the value unchanged."

Omission explicitly preserves the existing nickname under the current contract.

2. clears

Reviewer: "An empty string clears the nickname."

The empty string is the defined clearing input in this interface.

3. unspecified

Developer: "The meaning of null remains unspecified."

No agreed effect for null is supplied in the scenario.

4. nullable

Reviewer: "Optional does not automatically mean nullable."

Optional concerns absence, while nullable concerns allowed explicit values.

02 Briefing and language checks

Debugging from reproducible evidence

A Briefing checks / page 13

1. Which input fails?

B. A three-row file containing an empty date

Only the stated three-row example is established as failing.

2. What does the successful comparison change?

B. Both row count and content

Removing the row changes the number of rows and their content.

3. What does the stack trace establish?

D. The observed failure path reaches date parsing

The trace locates part of the failure path but does not isolate the cause.

D Language checks / page 18

1. Which conclusion is supported?

B. The observed failing path reaches date parsing

The trace supports a location statement, not the broader causal claims.

2. Which proposed comparison is strongest?

B. Keep three rows and other content fixed while varying the suspect date

Holding other factors fixed better isolates the condition being tested.

3. Which phrase avoids overstating evidence?

D. The empty date is a working hypothesis

A working hypothesis accurately labels the explanation as still requiring evidence.

4. What should the handoff preserve?

A. The failing fixture, conditions, and trace

Reproducible evidence needs the original input and execution context.

02 Conversation answers

02

Debugging from reproducible evidence

E Dialogue gaps

Pages 19-20

1. reproducible example. Turn 2. A reproducible example provides the input and conditions needed to observe the reported failure again.

2. hypothesis. Turn 4. Hypothesis labels the empty-date explanation as plausible but not yet isolated by a controlled test.

3. confounding variable. Turn 6. A confounding variable is another changed factor that prevents this comparison from isolating the date condition.

4. controlled comparison. Turn 8. Controlled comparison holds the other relevant conditions fixed while varying the suspected date condition.

5. input fixture. Turn 10. Input fixture names the prepared test data, which must remain available to reproduce the original failure.

6. stack trace. Turn 12. The stack trace records the observed call path around the error but does not alone establish its root cause.

7. failure signature. Turn 14. Failure signature distinguishes the original error pattern from a different failure in the comparison run.

8. deterministic behavior. Turn 16. Deterministic behavior concerns consistent results under the same relevant conditions, which repeated checks can investigate.

9. nonproduction environment. Turn 18. A nonproduction environment supports the investigation without treating this dialogue as permission to test on live workloads.

10. causal claim. Turn 20. A causal claim needs stronger evidence than the current confounded comparison, so the proposed cause remains qualified.

F Transfer conversation

Page 21

1. three

Developer: "The failing file contains three rows."

The supplied failing fixture has three rows, including one empty date.

2. two

Reviewer: "The successful file contains two rows."

Removing the suspect row leaves the stated successful two-row file.

3. hypothesis

Developer: "The empty date remains a hypothesis."

The current comparison has not isolated the date as the cause.

4. constant

Reviewer: "Keep the other conditions constant in the next comparison."

Holding other conditions constant helps isolate the suspected date effect.

03 Briefing and language checks

Giving actionable code-review feedback

03

A Briefing checks / page 22

1. What is the pull request titled?

B. Rename export helper

The stated title describes a helper rename, not a notification-policy change.

2. What additional change appears in the diff?

B. Notification moves outside the success branch

The changed call location permits notification after an unsuccessful export.

3. What part does Priya accept?

D. The naming change

Priya accepts the rename while objecting to the changed notification behavior.

D Language checks / page 27

1. Which comment is actionable?

A. Restore the success-only notification and test that failure sends no completion notice

The correct comment names the behavior, requested change, and verification.

2. Why is this not only a refactor?

A. Observable notification behavior changes

A behavior-preserving refactor would not add completion notices after failure.

3. What should the failure test assert?

B. No completion notice is sent

The defect concerns a false success message after export failure.

4. Which statement preserves Priya's position?

A. The rename is acceptable; the notification behavior needs correction

Priya distinguishes an acceptable naming change from an unresolved behavioral defect.

03 Conversation answers

Giving actionable code-review feedback

03

E Dialogue gaps

Pages 28-29

1. rename. Turn 2. Rename identifies the accepted identifier change, which does not explain or justify the additional behavioral change.

2. control flow. Turn 4. Control flow determines whether a failed export can reach the notification call after it moves outside the branch.

3. side effect. Turn 6. Side effect refers to the externally visible notification rather than the helper's new name.

4. failure path. Turn 8. The failure path must be exercised to check that unsuccessful exports do not trigger a completion notice.

5. review comment. Turn 10. A review comment is actionable when it identifies the changed location, effect, and requested correction.

6. review scope. Turn 12. Review scope should reflect the actual remaining changes rather than rely solely on the original rename title.

7. false success. Turn 14. False success is the completion indication produced even though the export operation failed.

8. patch revision. Turn 16. A patch revision is an updated proposed change, which still needs review rather than automatic approval.

9. regression test. Turn 18. A regression test should detect recurrence of the known false-notification defect rather than merely execute the helper.

10. verification evidence. Turn 20. Verification evidence supports the revised behavior and is needed before the reviewer can resolve the blocking concern.

F Transfer conversation

Page 30

1. name

Reviewer: "The helper name is acceptable."

Priya explicitly accepts the naming change in the pull request.

2. success

Author: "The notification must remain on the success path."

Completion notification is intended only for a successful export here.

3. failure

Reviewer: "Add a test for the failure path."

The regression occurs when export fails yet a completion notice appears.

4. approval

Author: "The current feedback is not yet approval."

The blocking behavior issue still requires correction and verification.

04 Briefing and language checks

Requesting dependency evidence

04

A Briefing checks / page 31

1. Which dependency does the application use directly?

B. ParcelKit

The application uses ParcelKit directly, while TextCore is introduced through ParcelKit.

2. What changes indirectly?

C. The resolved TextCore version

The proposed ParcelKit upgrade changes the TextCore version selected in the dependency resolution.

3. What does the note establish?

A. A parsing entry point is deprecated

Deprecation is stated, but usage, removal, and replacement compatibility remain unverified.

D Language checks / page 36

1. Which statement is accurate?

A. TextCore is transitive through ParcelKit

TextCore is brought into the application through its direct dependency ParcelKit.

2. What should be checked first about the selected version?

B. The resolved dependency record

The resolved record establishes the concrete TextCore version selected by this upgrade.

3. What does deprecated not establish?

B. That the entry point has been removed

Deprecation and removal are different statuses, and removal has not been established.

4. Which compatibility claim is justified now?

D. Compatibility remains unverified

Usage and replacement behavior have not yet been checked.

04 Conversation answers

Requesting dependency evidence

E Dialogue gaps

Pages 37-38

- 1. transitive dependency.** Turn 2. TextCore is a transitive dependency because the application receives it through ParcelKit rather than using it directly here.
- 2. lockfile.** Turn 4. The lockfile records resolved dependency selections and is more specific than a top-level version constraint alone.
- 3. deprecation.** Turn 6. Deprecation marks an interface for discouragement or transition but does not alone establish removal or current usage.
- 4. call site.** Turn 8. A call site shows where the affected parsing interface is invoked, which must be checked rather than assumed.
- 5. migration guide.** Turn 10. The migration guide provides transition details that a similar replacement name cannot establish.
- 6. semantic versioning.** Turn 12. Semantic versioning provides specific conventions only when the relevant project actually adopts and follows that policy.
- 7. dependency tree.** Turn 14. The dependency tree makes the ParcelKit-to-TextCore relationship visible so indirect impact is included in the review.
- 8. public interface.** Turn 16. The public interface is the exposed parsing surface whose supported behavior and affected callers need verification.
- 9. upgrade evidence.** Turn 18. Upgrade evidence combines actual versions, affected usage, documentation, and relevant checks rather than a broad assertion of safety.
- 10. compatibility.** Turn 20. Compatibility remains unverified until the relevant consumers and behavior have been checked against the proposed versions.

F Transfer conversation

Page 39

1. direct

Developer: "ParcelKit is the direct dependency."

The application uses ParcelKit directly under the stated dependency relationship.

2. transitive

Reviewer: "TextCore is the transitive dependency."

TextCore is introduced through ParcelKit rather than directly here.

3. deprecated

Developer: "The parsing entry point is marked deprecated."

The change note establishes deprecation, not removal or usage.

4. unverified

Reviewer: "Replacement compatibility remains unverified."

The relevant behavior and affected callers have not yet been checked.

05 Briefing and language checks

Explaining what a test actually exercises

A Briefing checks / page 40

1. What does the storage double currently do?

A. Always succeeds

The supplied test double is configured only for successful storage.

2. What did the previous bug show?

C. Success after storage rejected the save

The prior defect was a false success indication after rejection.

3. What remains separately untested?

C. The real storage adapter

The scenario explicitly states that the real adapter has not been tested.

D Language checks / page 45

1. Why does the current test miss the prior bug?

C. Its double never rejects

The failing condition required to expose the bug is absent from the fixture.

2. Which addition targets the defect?

C. Configure rejection and assert no success message

The new case must trigger rejection and verify the missing user-visible boundary.

3. What does the rejection unit test not establish?

A. Correct behavior of the real storage adapter

The test still substitutes for the real adapter rather than exercising it.

4. Which report is precise?

B. Success-path coverage exists; rejection and real-adapter coverage need work

The report separates existing coverage from two distinct unresolved areas.

05 Conversation answers

Explaining what a test actually exercises

E Dialogue gaps

Pages 46-47

- 1. happy path.** Turn 2. The happy path is the successful execution route supplied by the always-successful storage double.
- 2. test double.** Turn 4. The test double replaces real storage and must be configured to produce the rejection needed by this test.
- 3. negative assertion.** Turn 6. A negative assertion checks that the unwanted success message does not appear after the simulated rejection.
- 4. Interaction verification.** Turn 8. Interaction verification checks dependency calls but does not alone establish the correct user-visible result.
- 5. integration test.** Turn 10. An integration test would exercise the relevant connected components, whereas this case still replaces the real storage adapter.
- 6. coverage gap.** Turn 12. The coverage gap for the real adapter remains open even if the simulated rejection unit case passes.
- 7. arrange.** Turn 14. Arrange is the preparation stage that configures the rejection condition before the save behavior is executed.
- 8. success response.** Turn 16. Success response is the outcome configured in the existing test, so its evidence is limited to that path.
- 9. behavioral verification.** Turn 18. Behavioral verification checks the observed application result, while retaining the boundary imposed by the substituted dependency.
- 10. unit test.** Turn 20. A unit test can cover the application's rejection handling without proving the behavior of the real storage integration.

F Transfer conversation

Page 48

1. succeeds

Developer: "The existing double always succeeds."

The current fixture supplies only successful storage outcomes to the save operation.

2. rejection

Reviewer: "The new case must trigger rejection."

Rejection is the missing condition that exposed the previous false-success bug.

3. success

Developer: "The assertion should prevent a false success message."

The known defect incorrectly displayed success after storage rejected the save.

4. untested

Reviewer: "The real adapter remains untested."

Simulated rejection coverage does not exercise the real storage adapter.

05

06 Briefing and language checks

Comparing performance tradeoffs fairly

A Briefing checks / page 49

1. Which conditions are matched?

B. Input and machine

The benchmark explicitly uses the same input and machine for both implementations.

2. How far is B over the memory budget?

C. 40 megabytes

B uses 160 megabytes against a budget of 120, exceeding it by forty.

3. What is unmeasured?

B. Tail latency and production behavior

Only the benchmark median and memory figures are supplied, not tail or production results.

D Language checks / page 54

1. What is B's median reduction relative to A?

B. 37.5 percent

Fifteen divided by the forty-millisecond baseline equals 0.375, or 37.5 percent.

2. Which budget statement is correct?

C. A is 40 below and B is 40 above

The 120-megabyte limit lies forty above A and forty below B.

3. Which claim exceeds the evidence?

D. B improves every production request

Production behavior and every-request outcomes have not been measured.

4. Which summary is balanced?

D. B lowers median latency but exceeds the memory budget; tail and production remain unmeasured

The summary combines the measured benefit, resource constraint, and remaining evidence limits.

06 Conversation answers

Comparing performance tradeoffs fairly

E Dialogue gaps

Pages 55-56

- 1. benchmark.** Turn 2. Benchmark identifies the controlled measurement, whose conditions limit the scope of the comparison.
- 2. memory footprint.** Turn 4. Memory footprint is the measured memory usage, which is 160 megabytes for B against a 120-megabyte budget.
- 3. relative reduction.** Turn 6. Relative reduction divides the fifteen-millisecond decrease by A's forty-millisecond baseline, yielding 37.5 percent.
- 4. absolute difference.** Turn 8. Absolute difference is the direct fifteen-millisecond reduction, which can be reported alongside the relative latency reduction.
- 5. Tail latency.** Turn 10. Tail latency concerns slower-end observations and cannot be inferred from the supplied median alone.
- 6. headroom.** Turn 12. Headroom is the forty-megabyte difference between A's eighty-megabyte usage and the 120-megabyte limit.
- 7. Production behavior.** Turn 14. Production behavior remains unmeasured, so the controlled benchmark cannot establish live-system outcomes.
- 8. tradeoff.** Turn 16. Tradeoff connects B's lower median latency with its higher memory use and failure to meet the stated budget.
- 9. budget compliance.** Turn 18. Budget compliance is evaluated against the stated 120-megabyte limit, which B exceeds regardless of its latency gain.
- 10. controlled conditions.** Turn 20. Controlled conditions specify the matched input and machine and prevent the result being generalized without further evidence.

F Transfer conversation

Page 57

1. 25

Developer: "B's median is 25 milliseconds."

The measured median for implementation B is twenty-five milliseconds.

2. 15

Reviewer: "The median decreases by 15 milliseconds relative to A."

Forty minus twenty-five gives a fifteen-millisecond absolute decrease.

3. 160

Developer: "B uses 160 megabytes of memory."

The supplied memory measurement for implementation B is 160 megabytes.

4. 40

Reviewer: "B exceeds the budget by 40 megabytes."

One hundred sixty minus the 120-megabyte budget leaves forty over.

07 Briefing and language checks

Discussing retries and shared state

A Briefing checks / page 58

1. What did the client observe?

B. No reply before its timeout

A timeout establishes lack of a timely reply, not the service's final action.

2. What might the service have done?

A. Created the reservation already

The scenario explicitly leaves prior creation possible and unresolved.

3. What is unspecified?

D. Duplicate-handling behavior for the proposed retry

The service's treatment of duplicate requests is not defined in the supplied contract.

D Language checks / page 63

1. Which statement follows from the timeout?

C. The client did not receive a timely reply

A client timeout leaves the service-side result uncertain.

2. Why is a new reservation request risky?

B. The first may already have created a reservation

A repeated new operation may add another effect while the first outcome remains unknown.

3. What does an idempotency key require?

C. Agreed service semantics

A key is useful only when the service contract defines how repeated intent is handled.

4. Which next step is appropriate?

D. Clarify reconciliation and retry behavior, then test it outside production

The contract and controlled evidence are missing, and production changes are not authorized.

07 Conversation answers

Discussing retries and shared state

E Dialogue gaps

Pages 64-65

- 1. uncertain outcome.** Turn 2. Uncertain outcome means the first reservation may exist even though the client did not receive a timely response.
- 2. timeout.** Turn 4. Timeout describes expiration of the client's wait, not a confirmed service-side rejection or absence of a reservation.
- 3. deduplication.** Turn 6. Deduplication behavior is unspecified, so repeated intent cannot be assumed to produce only one reservation.
- 4. idempotency key.** Turn 8. An idempotency key needs a defined service contract for repeated intent rather than merely being added to a request.
- 5. request identity.** Turn 10. Request identity distinguishes a repeated attempt at the original operation from a new reservation with another intended effect.
- 6. status lookup.** Turn 12. Status lookup is a possible reconciliation mechanism that must actually be supported and defined rather than invented.
- 7. Backoff.** Turn 14. Backoff changes timing between attempts but does not itself establish the earlier outcome or prevent duplicate business effects.
- 8. authorization.** Turn 16. Authorization is explicitly absent for production changes, so investigation must not be described as permission to enable the retry.
- 9. shared state.** Turn 18. Shared state is the reservation state whose service-side result may differ from what the timed-out client knows.
- 10. reconciliation.** Turn 20. Reconciliation establishes the actual first-request outcome and supports a defined retry decision rather than assuming failure from silence.

F Transfer conversation

Page 66

1. timeout

Developer: "The client observed a timeout."

The client stopped waiting before it received the service reply.

2. exist

Reviewer: "The first reservation may already exist."

The service may have completed creation before the client timed out.

3. unspecified

Developer: "Duplicate handling remains unspecified."

The service contract supplies no duplicate-handling behavior for this proposal.

4. authorized

Reviewer: "No production change is authorized."

The scenario explicitly withholds permission to enable a live change.

08 Briefing and language checks

Handing over a release with explicit limits

A Briefing checks / page 67

1. Where is R18 deployed?

D. Staging

The scenario establishes a staging deployment only, not production deployment.

2. What is the new search feature's state?

D. Disabled by a feature flag

The search feature is present but disabled by its flag.

3. Which check is unresolved?

A. Compatibility with the previous application version

Compatibility with the prior application has not been tested.

D Language checks / page 72

1. Which status is accurate?

C. R18 is in staging with search disabled

Only the staging deployment and disabled search state are established.

2. Why does optional not settle compatibility?

D. The previous application's behavior with migrated data is untested

Optional describes a field rule, not evidence of the older application's behavior.

3. Which rollback claim is justified?

D. Readiness remains unverified without the relevant compatibility evidence

The earlier application's compatibility with the changed state has not been tested.

4. Who owns the unresolved production decision?

C. Lee

Lee is explicitly identified as the reviewer responsible for the production decision.

08 Conversation answers

Handing over a release with explicit limits

E Dialogue gaps

Pages 73-74

- 1. environment boundary.** Turn 2. Environment boundary limits the deployment claim to staging rather than implying a live production release.
- 2. feature flag.** Turn 4. The feature flag controls whether new search is enabled, which is separate from the code being deployed.
- 3. known limitation.** Turn 6. Known limitation identifies the untested previous-version compatibility rather than presenting optionality as proof.
- 4. Schema compatibility.** Turn 8. Schema compatibility requires relevant evidence about components working with the changed structure, which is not supplied here.
- 5. rollback.** Turn 10. Rollback is a return to an earlier state whose readiness depends on compatibility with the actual migrated data and system state.
- 6. data compatibility.** Turn 12. Data compatibility concerns whether the earlier application can correctly use the actual migrated state, not whether an old executable exists.
- 7. go/no-go decision.** Turn 14. The go/no-go decision is Lee's unresolved production decision, not an automatic consequence of receiving the handoff.
- 8. handoff.** Turn 16. Handoff transfers specific state, evidence gaps, and responsibility so the reviewer does not have to infer readiness.
- 9. release gate.** Turn 18. A release gate is a condition for advancing the release and cannot be treated as satisfied by an unrelated completed deployment.
- 10. readiness claim.** Turn 20. A readiness claim must stay within the completed checks and unresolved decision rather than equating staging deployment with approval.

F Transfer conversation

Page 75

1. staging

Developer: "R18 is deployed in staging."

The only established deployment environment in the scenario is staging.

2. disabled

Reviewer: "New search remains disabled."

The feature flag keeps search disabled despite the completed deployment.

3. untested

Developer: "Prior-version compatibility is untested."

No test of the previous application against the migrated state is established.

4. Lee

Reviewer: "The production decision remains with Lee."

Lee owns the production review and has not made that decision.

08

09 Conversation answers

The newer tag cannot repair an older payload

E Extended conversation

Pages 76-77

1. stale representation. Turn 2. B edited the older v7 representation containing eighteen dollars; the current representation is v8 with twenty dollars, so the submitted full record is stale.

2. If-Match precondition. Turn 4. If-Match requires the supplied entity tag to match the current representation using strong comparison; v7 does not match the current v8 here.

3. lost update. Turn 6. Sending the stale eighteen-dollar price with a current tag could overwrite A's twenty-dollar change; replacing only the tag does not reconcile the old body.

4. strong ETag. Turn 8. A strong ETag is the representation validator used by this conditional request; it is not permission to submit stale field values or proof of user intent.

5. HTTP 412. Turn 10. HTTP 412 is the specified precondition-failure response in this case; it does not establish a server outage, a successful save, or a generic authorization failure.

6. conflict reconciliation. Turn 12. Conflict reconciliation compares the current record with the intended edit before constructing another request; copying v8 onto the old body would bypass that reasoning.

F Transfer exchange

Page 78

1. 412

Developer: The returned status is 412 .

The case explicitly specifies precondition failure; the stale v11 tag does not match v12.

2. 9

Reviewer: The stored quantity remains 9 .

The rejected request makes no change, so the current quantity nine is preserved.

3. old

Developer: The stored note is still old .

The requested new note was not applied when the conditional request failed.

4. stale

Reviewer: Replacing only the tag would leave the payload stale .

The body would still contain seven rather than the current nine; a newer tag alone does not reconcile those values.

09

Background reading: [RFC 9110, section 13.1.1: If-Match and lost-update protection](#). The conversation is an original fictional teaching case.

10 Conversation answers

Fifty orders, fifty-one queries

E Extended conversation

Pages 79-80

- 1. N+1 query pattern.** Turn 2. One initial order query plus one line query for each of fifty orders produces fifty-one queries; the extra work grows with the number of orders.
- 2. lazy loading.** Turn 4. Lazy loading defers fetching related data until access; in this supplied trace each order's line collection triggers another query.
- 3. batched lookup.** Turn 6. The proposed implementation uses one related-line query for the fifty selected order IDs, replacing fifty separate line queries while retaining the initial order query.
- 4. round-trip count.** Turn 8. The new path has two serial round trips, contributing eight milliseconds in the supplied model; the count is not the complete application response time.
- 5. network component.** Turn 10. Eight milliseconds accounts only for the two modeled network delays; database execution, transferred data, and application processing remain additional and unmeasured.
- 6. result set.** Turn 12. The result set is the data returned by the queries; reducing query count does not establish its size, memory cost, or correct association with each order.

F Transfer exchange

Page 81

- 1. 5**
Developer: The batched path makes 5 total queries.
Four groups of fifty IDs require four line queries plus one initial order query, totaling five.
- 2. 20**
Reviewer: Its modeled network delay is 20 milliseconds.
Five serial round trips multiplied by four milliseconds equals twenty milliseconds, excluding all other work.
- 3. 201**
Developer: The old path makes 201 total queries.
One order query plus two hundred individual line queries equals two hundred one.
- 4. 784**
Reviewer: The modeled network-delay reduction is 784 milliseconds.
The old component is 201 times 4, or 804 milliseconds; subtract the new 20 to obtain 784, not a measured total-response improvement.

10

Background reading: [SQLAlchemy documentation: lazy loading, N+1 queries, and select-in loading](#). The conversation is an original fictional teaching case.

11 Conversation answers

One cent, two different contracts

E Extended conversation

Pages 82-83

1. rounding stage. Turn 2. The agreed contract rounds each line's charge before summing; rounding once on the basket subtotal is a different calculation even with the same rate.

2. half-up rule. Turn 4. For these positive amounts, half-up rounds the exact half-cent tie upward to one cent; it must not be replaced with half-even or an unspecified default.

3. basket-level calculation. Turn 6. The proposed calculation rounds 0.015 once to 0.02 and totals 0.17; it is internally consistent but does not implement the supplied line-level contract.

4. decimal arithmetic. Turn 8. Decimal arithmetic can preserve these exact decimal inputs and the requested rounding operation; merely formatting a binary result does not implement the missing line-level rule.

5. rounding mode. Turn 10. The rounding mode determines how ties are resolved; the contract explicitly requires half-up rather than relying on a library's possibly different default.

6. line-item boundary. Turn 12. The three separate line items each receive their own rounded charge; merging them into one line would change the supplied contract's calculation unit.

F Transfer exchange

Page 84

1. 0.03

Developer: Each rounded line charge is 0.03 dollars.

Ten percent of 0.25 is exactly 0.025; half-up to two decimal places yields 0.03.

2. 0.12

Reviewer: The sum of the four rounded charges is 0.12 dollars.

Four separate charges of 0.03 total 0.12; do not round the basket instead.

3. 1.12

Developer: The contract's final total is 1.12 dollars.

The four prices total 1.00, to which the line-level charge total of 0.12 is added.

4. 0.02

Reviewer: That exceeds the basket-level total by 0.02 dollars.

The basket method gives 1.00 plus 0.10, or 1.10; the contract total 1.12 is two cents higher.

11

Background reading: [Python documentation: exact decimal inputs, quantize, and rounding modes](#). The conversation is an original fictional teaching case.

Find the move you need

Use these as precise language models. Replace details only when you know the actual facts, authority, and agreed conditions.

01 Clarifying interface contracts

Name the boundary: I want to confirm the profile interface contract.

Identify the proposal: The proposed client sends null.

Avoid borrowing a rule: I would not infer this from another API.

Full phrase set: pages 7-8 | Dialogue: pages 10-11

02 Debugging from reproducible evidence

Report the failing input: The three-row file fails during import.

Report the comparison: The two-row file succeeds.

Propose a narrow change: Change only the suspect date condition.

Full phrase set: pages 16-17 | Dialogue: pages 19-20

03 Giving actionable code-review feedback

Acknowledge the acceptable part: The helper name is clear.

Separate preference and defect: My concern is behavior, not the naming choice.

Request evidence: Add a test for a failed export.

Full phrase set: pages 25-26 | Dialogue: pages 28-29

04 Requesting dependency evidence

Identify the direct dependency: The application uses ParcelKit directly.

Request the actual selection: Which version does the lockfile record?

Ask about usage: Do we have any affected call sites?

Full phrase set: pages 34-35 | Dialogue: pages 37-38

Keep the conversation productive

Use these as precise language models. Replace details only when you know the actual facts, authority, and agreed conditions.

05 Explaining what a test actually exercises

Describe the current setup: The storage double always succeeds.

Identify the gap: The current test never triggers rejection.

Separate call and result: A storage call alone does not prove the right user message.

Full phrase set: pages 43-44 | Dialogue: pages 46-47

06 Comparing performance tradeoffs fairly

Name the conditions: Both runs use the same input and machine.

Report the absolute gain: B lowers the median by fifteen milliseconds.

Calculate the overrun: B exceeds that budget by forty megabytes.

Full phrase set: pages 52-53 | Dialogue: pages 55-56

07 Discussing retries and shared state

State the observation: The client timed out before receiving a reply.

Identify the proposal: The retry would submit a new reservation request.

Clarify identity: Would this carry the same request identity or a new one?

Full phrase set: pages 61-62 | Dialogue: pages 64-65

08 Handing over a release with explicit limits

Identify the release: This handoff concerns R18.

Separate deployment and use: Deployed does not mean enabled.

Name the decision owner: Lee owns the production review.

Full phrase set: pages 70-71 | Dialogue: pages 73-74

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

absolute difference report the absolute difference	51
acceptance criterion state an acceptance criterion	6
acknowledgment distinguish an acknowledgment	59
act act on the save request	42
adapter test the storage adapter	42
application programming interface document the API	5
approval withhold approval pending correction	24
arrange arrange a rejection case	42
assert assert no false success	42
assertion strengthen the assertion	24
assertion add a negative assertion	41
at-least-once delivery discuss at-least-once delivery	59
at-most-once delivery distinguish at-most-once delivery	60
atomic operation define the atomic operation	60
authorization obtain production authorization	60
backoff configure retry backoff	60
backward compatibility assess backward compatibility	6
backward compatibility verify backward compatibility	69
baseline use A as the baseline	50
batched lookup replace repeated calls with a batched lookup	15
behavior change flag the behavior change	23
behavioral verification perform behavioral verification	42
benchmark run a benchmark	50
blocking concern explain the blocking concern	24

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

bottleneck identify the bottleneck	51
branch coverage examine branch coverage	42
breaking change identify a breaking change	33
budget compliance check budget compliance	51
call site identify the call site	23
call site search for affected call sites	32
causal claim qualify a causal claim	15
changelog compare the changelog	33
compatibility verify compatibility	33
completion notification gate the completion notification	23
concurrency account for concurrency	60
confounding variable remove a confounding variable	14
consumer identify the consumer	6
contract test add a contract test	6
control flow trace the control flow	23
controlled comparison run a controlled comparison	14
controlled conditions preserve controlled conditions	51
correlation identifier retain the correlation identifier	60
coverage gap report the coverage gap	42
data compatibility test data compatibility	69
date parsing investigate date parsing	14
decimal arithmetic use decimal arithmetic	6
deduplication define deduplication behavior	60
default value document the default value	5

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

dependency tree inspect the dependency tree	32
deployment report the deployment	68
deprecation investigate a deprecation	32
deserialization verify deserialization behavior	6
deterministic behavior check deterministic behavior	15
diff inspect the diff	23
direct dependency upgrade a direct dependency	32
disabled state preserve the disabled state	68
distribution inspect the latency distribution	51
duplicate request detect a duplicate request	59
duplicate side effect prevent a duplicate side effect	59
empty field isolate the empty field	15
empty string send an empty string	5
entry point locate the parsing entry point	32
environment boundary preserve the environment boundary	69
environment parity establish environment parity	15
error path exercise the error path	42
exception capture the exception	15
exception configure an exception	41
failure path cover the failure path	23
failure signature compare the failure signature	15
false negative avoid a false negative	42
false success prevent false success	24
feature flag verify the feature flag	68

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

field presence check field presence	6
fixture arrange the fixture	41
forward compatibility check forward compatibility	69
go/no-go decision record the go/no-go decision	69
guard condition restore the guard condition	24
handoff prepare the release handoff	69
happy path cover the happy path	42
headroom calculate memory headroom	50
hypothesis test a hypothesis	14
idempotency specify idempotency semantics	59
idempotency key define an idempotency key	59
input fixture preserve the input fixture	14
instrumentation improve instrumentation	15
integration test add an integration test	41
intent confirm the intent	24
interaction verification separate interaction verification	42
interface contract clarify the interface contract	5
known limitation list known limitations	69
latency measure latency	50
lockfile inspect the lockfile	32
major version interpret the major version	33
malformed input identify malformed input	15
measurement noise account for measurement noise	51
median report median latency	50

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

memory footprint compare the memory footprint	50
migration review the migration	68
migration guide consult the migration guide	32
minor version check minor-version guarantees	33
mock inspect mock interactions	41
N+1 query pattern identify an N+1 query pattern	15
negative assertion verify a negative assertion	41
nonblocking suggestion label a nonblocking suggestion	24
nonproduction environment use a nonproduction environment	15
null value define the null value	5
nullable field declare a nullable field	5
observable behavior preserve observable behavior	24
omitted field distinguish an omitted field	5
optimistic concurrency enforce optimistic concurrency	14
optional field omit an optional field	5
optional field add an optional field	68
outlier investigate an outlier	51
overflow quantify the budget overflow	50
p95 distinguish p95 from the median	50
parser inspect the parser	14
partial update apply a partial update	5
patch revision request a patch revision	24
patch version verify patch-version policy	33
percentile report a latency percentile	50

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

pin review the version pin	33
previous version test the previous version	68
producer align producer expectations	6
production authorize production release	68
production behavior avoid assuming production behavior	51
public interface compare the public interface	33
pull request review the pull request	23
race condition investigate a race condition	60
readiness claim qualify the readiness claim	69
reconciliation request outcome reconciliation	59
refactor separate a refactor from a behavior change	23
regression investigate a regression	15
regression test add a regression test	24
rejection simulate a rejection	41
relative reduction calculate the relative reduction	51
release candidate review the release candidate	68
release gate satisfy the release gate	69
release note read the release note	32
release owner name the release owner	69
removal verify removal separately	32
rename approve the rename	23
replacement interface assess the replacement interface	33
reproducible example share a reproducible example	14
reproduction obtain a reproduction	14

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

request identity preserve request identity	59
request payload inspect the request payload	5
resolution reproduce dependency resolution	33
resolved version record the resolved version	32
resource budget respect the resource budget	50
retry define retry behavior	59
retry budget set a retry budget	60
review comment write a review comment	23
review scope clarify review scope	24
rollback assess rollback readiness	68
rollout plan the rollout	68
root cause isolate the root cause	14
rounding mode specify the rounding mode	14
sample document the sample	51
sandbox test in a sandbox	60
schema revise the schema	6
schema compatibility verify schema compatibility	68
semantic versioning confirm semantic versioning policy	33
serialization inspect serialization behavior	6
shared state reconcile shared state	59
side effect examine the side effect	23
sign-off obtain release sign-off	69
stack trace inspect the stack trace	14
staging deploy to staging	68

Find a term and its context

The page number leads to the definition and a typical word combination. Repeated terms have separate entries when they serve different lesson contexts.

status lookup define a status lookup	60
stored state preserve stored state	5
strong ETag compare the strong ETag	6
stub use a success stub	41
success branch keep the call in the success branch	23
success response return a success response	41
tail latency measure tail latency	50
test double configure the test double	41
test isolation preserve test isolation	41
test oracle define the test oracle	42
throughput measure throughput separately	50
timeout observe a timeout	59
tradeoff explain the tradeoff	51
transaction boundary clarify the transaction boundary	60
transitive dependency inspect a transitive dependency	32
uncertain outcome report an uncertain outcome	59
unit test extend the unit test	41
unspecified interface behavior resolve unspecified interface behavior	6
upgrade evidence assemble upgrade evidence	33
validation rule agree on a validation rule	5
verification evidence provide verification evidence	24
verification matrix update the verification matrix	69
version constraint check a version constraint	32
workload describe the workload	51

Original cases. Clear boundaries.

These are original fictional teaching conversations, not transcripts of real workers. Names, figures, organizations, and incidents are invented. Sources support terminology and communication aims, not the fictional results or an endorsement of this book.

[1] Google Engineering Practices. What to Look for in a Code Review.

[Open the original source](#)

Context for reviewing functionality, tests, naming, and changed behavior. All pull requests and conversations here are original. Checked 10 October 2026.

[2] Amazon Builders Library. Making Retries Safe with Idempotent APIs.

[Open the original source](#)

Background on uncertain outcomes and retry contracts. The fictional reservation interface is not an AWS service or implementation guide. Checked 10 October 2026.

[3] Python Documentation. unittest.mock.

[Open the original source](#)

Reference for configured test doubles, return values, and raised exceptions. Exercises distinguish simulated behavior from actual dependency coverage. Checked 10 October 2026.

[4] Semantic Versioning 2.0.0.

[Open the original source](#)

Terminology for public interfaces, compatibility, and deprecation. The fictional packages are not assumed to follow this versioning convention. Checked 10 October 2026.

Phil Stilwell

© 2026 Phil Stilwell. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission from the author, except as permitted by law.

All products, people, measurements, interfaces, and release processes are fictional. These scenarios teach professional English, not production operating instructions. Actual contracts, security requirements, engineering review, test environments, and change authorization take precedence. No exercise authorizes production changes or guarantees correctness, compatibility, reliability, or performance. Code-review and testing terminology is used without requiring a particular language or framework.

English Ladder | English for Work | 2026 edition. The B2-C1 range is an editorial study recommendation, not a proficiency certification.

[Open the matching English Ladder course](#)